WNDRVR
SUPPORT NETWORK

VXWORKS 7 IKE PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 IKE Programmer's Guide, SR0650

16 September 2020

1. WIND RIVER IKE OVERVIEW

Wind River IKE is a target-and-RTOS portable implementation of the Internet Key Exchange (IKE) protocols specified by the Internet Engineering Task Force (IETF).

When deployed with an implementation of IP security (IPsec) such as Wind River IPsec, it negotiates the security associations required for IPsec to protect data traffic between network peers.

Wind River IKE interoperates with other RFC-compliant implementations of IKE; for example, those available on Windows XP, Windows Vista, Linux, BSD variants, and on Cisco equipment. It is also designed to work with a PF_KEY v2-compliant implementation of IPsec. Wind River acknowledges the OpenBSD project for their PF_KEY v2 extensions.

This guide outlines the IKE technologies and how they are implemented in Wind River IKE. It contains instructions for building and configuring a VxWorks® executable that includes Wind River IKE. It explains how to further configure Wind River IKE at run time using a configuration file, API calls, and IPCOM shell commands. Example implementations are provided.

Technology Overview on page 1 Internet Key Exchange (IKE) is a hybrid protocol used to establish secure communication between network peers, specified in RFC 2409.
Product Overview on page 1 Wind River IKE is a target- and RTOS-portable implementation of the Internet Key Exchange (IKE) protocols specified by the Internet Engineering Task Force (IETF).

1. Technology Overview

Internet Key Exchange (IKE) is a hybrid protocol used to establish secure communication between network peers, specified in RFC 2409.

The Internet Key Exchange (IKE) and RFC 4306: Internet Key Exchange (IKEv2) Protocol. IKE provides the following services:

- mutual authentication between two peers
- negotiation of protection suites for IPsec

For more information on the technologies underlying the above capabilities, see *VxWorks 7 IKE Programmer's Guide: Technology Background on page 4*.

Parent topic: Wind River IKE Overview on page 1

2. Product Overview

Wind River IKE is a target- and RTOS-portable implementation of the Internet Key Exchange (IKE) protocols specified by the Internet Engineering Task Force (IETF).

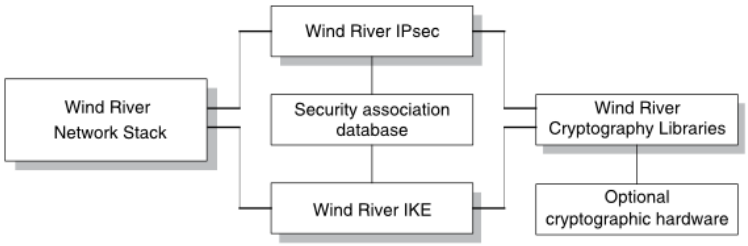
Wind River IKE implements the RFCs listed in RFCs on page 2.

In a network that employs IP data security, the Wind River Network Stack passes IP packets to Wind River IPsec, which encrypts or decrypts the packets before returning them to the stack. If required, the instance of IPsec running on a packet's source host negotiates a cryptographic protection suite with the packet's destination host, using an automated process such as that provided by Wind River IKE.

The Wind River Cryptography Libraries supply cryptographic algorithm support to VxWorks applications. Wind River products such as Wind River IKE call the functions as needed. In addition, other applications can call the Cryptography Libraries functions using the standard OpenSSL API.

The relationships between Wind River IKE and other Wind River products are expressed in the following diagram.

Figure 1. Product Relationships



For more information on how Wind River IKE works, see *VxWorks 7 IKE Programmer's Guide:* *Technical Background on page 4*.

For more information on Wind River IPsec, see the *VxWorks 7 IPsec Programmer's Guide*.

For more information on Wind River Cryptography Libraries, see the *VxWorks 7 Cryptography Libraries Programmer's Guide*.

RFCs on page 2 Wind River IKE complies with numerous IETF RFCs.
Additional Documentation on page 3 Other sources of information about Wind River IKE are available.

Parent topic: Wind River IKE Overview on page 1

2.1. RFCs

Wind River IKE complies with numerous IETF RFCs.

Wind River IKE complies with the following IETF RFCs, except where noted. The RFCs are accessible from the following Web site:

http://www.ietf.org

IKE Protocols

- RFC 2407: *The Internet IP Security Domain of Interpretation for ISAKMP*
  - Section 4.4.1.4: PROTO_IPCOMP is not implemented.
  - Section 4.4.4.8: ESP_3IDEA is not implemented.
  - Section 4.4.4.9: ESP_DES_IV32 is not implemented.
- RFC 2408: *Internet Security Association and Key Management Protocol (ISAKMP)*
- RFC 2409: *The Internet Key Exchange (IKE)*
  - Section 6.3: Third Oakley Group is not implemented.
  - Section 6.4: Fourth Oakley Group is not implemented.
- RFC 3706: *A Traffic-Based Method of Detecting Dead Internet Key Exchange (IKE) Peers*

- RFC 3947: *Negotiation of NAT-Traversal in the IKE*
- RFC 4301: *Security Architecture for the Internet Protocol*
- RFC 4304: *Extended Sequence Number (ESN) Addendum to IPsec Domain of Interpretation (DOI) for Internet Security Association and Key Management Protocol (ISAKMP)*
- RFC 4306: *Internet Key Exchange (IKEv2) Protocol*
  - Section 2.22: IPComp is not implemented.
- RFC 4555: *IKEv2 Mobility and Multihoming Protocol (MOBIKE)*
- RFC 4621: *Design of the IKEv2 Mobility and Multihoming (MOBIKE) Protocol*
- draft-ietf-ipsec-isakmp-mode-cfg-05.txt The ISAKMP Configuration Method
- RFC 4718: *IKEv2 Clarifications and Implementation Guidelines*
- RFC 4806: *Online Certificate Status Protocol (OCSP) Extensions to IKEv2*

  RFC 5857: *IKEv2 Extensions to Support Robust Header Compression over IPsec*

Cryptographic Algorithms

- RFC 3664: *The AES-XCBC-PRF-128 Algorithm for the Internet Key Exchange Protocol (IKE)*
- RFC 4307: *Cryptographic Algorithms for Use in the Internet Key Exchange Version 2 (IKEv2)*
- RFC 4109: *Algorithms for Internet Key Exchange version 1 (IKEv1)*
  - Section 4: Summary RSA with encryption is not implemented. Tiger for hashing is not implemented. D-H Group 14 (2048) is not implemented. D-H Elliptic Curves is not implemented.
- RFC 4868: *Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec*

Parent topic: Product Overview on page 1

2.2. Additional Documentation

Other sources of information about Wind River IKE are available.

The following documents provide information about products related to Wind River IKE:

- VxWorks 7 IPsec Programmer's Guide
- VxWorks 7 Cryptography Libraries Programmer's Guide
- *Wind River RADIUS and Diameter for VxWorks 7 Programmer's Guide*
- *Wind River EAP for VxWorks 7 Programmer's Guide*

OpenBSD

Licensing and other information about OpenBSD is available at the OpenBSD Project Web site: http://www.openbsd.org/policy.html

Parent topic: Product Overview on page 1

2. TECHNOLOGY BACKGROUND

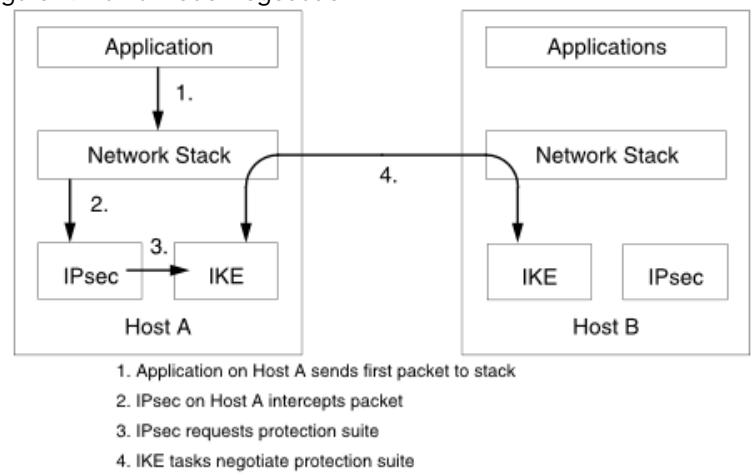
1. How IKE Works

Wind River IKE implements the requirements of RFC 2409: The Internet Key Exchange (IKE), RFC 4306: Internet Key Exchange (IKEv2) Protocol.

Other specifications in RFCs on page 2 are also supported.

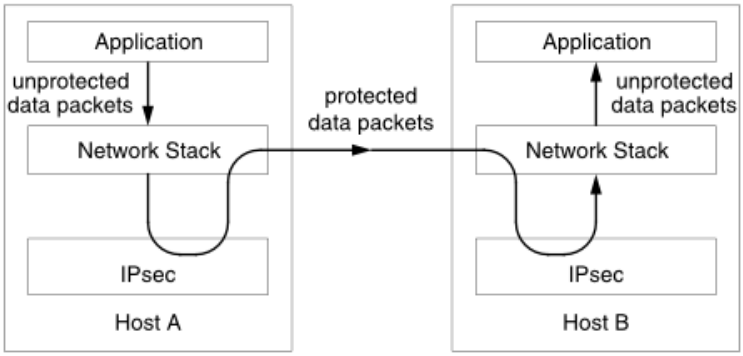
IKE is a hybrid protocol used to create secure communication channels between network peers. It does so by negotiating security associations (SAs), which establish the protection suites to be used when IPsec protects data traffic.

Figure 2. Traffic-mode Negotiation



The diagram above shows the steps of traffic-mode IKE negotiation; that is, negotiation initiated by the flow of data to IPsec. (IKE can also be configured in initiator mode, in which negotiation begins as soon as the IKE task starts.)

Figure 3. IPsec Data Flow



When a Host A application first sends an IP packet to Host B by means of the IPsec-enabled network stack, IPsec intercepts the packet and issues a request for a protection suite. Host A's IKE task then communicates with Host B's IKE task through the stack. The two tasks negotiate the protection suite to be used for the first packet and for subsequent packets of the same type.

IPsec then handles the traffic as shown in the diagram *IPsec Data Flow* . IKE has no further involvement until IPsec again requests a protection suite; for example, to handle a new packet type.

Security Associations on page 5 An SA is a contract between two network peers, negotiated by IKE or set manually, that governs how the peers protect data passing between them.
Authentication Using a Preshared Key on page 6 Network peers can establish trust in each other through the use of a preshared key (also called a preshared secret).
Authentication Using Certificates on page 6 Network peers can establish trust in each other through the exchange of X.509 certificates.
Perfect Forward Secrecy on page 7 Perfect Forward Secrecy protects against the compromise of a private or current session key putting earlier or subsequent keys at risk.
Flows on page 7 In Wind River IKE, flows act as unique identifiers for sets of IPsec SA negotiation parameters (in the IKE configuration file) and for individual SAs (in the SADB).

1.1. Security Associations

An SA is a contract between two network peers, negotiated by IKE or set manually, that governs how the peers protect data passing between them.

An SA applies to packets of a specific traffic type passing between specific ports on hosts at specific addresses. It determines the security protocol (AH or ESP), protocol mode, and protocol options to be used by IPsec for those packets. It is uniquely identified by a 4-tuple consisting of a Security Parameter Index (SPI), the source address, the destination address, and the protocol.

For typical bidirectional communication between two hosts or between two security gateways, two SAs are required, one for each direction.

SA Administration

Wind River IKE includes the ipike_config_api functions for creating and modifying IKE SA and IPsec SA negotiation parameters in the IKE configuration.

When you update IKE SA or IPsec SA negotiation parameters, whether by means of the configuration file, the ike config shell command, or an ipike_config_api function, Wind River IKE deletes the corresponding SA from the Security Associations Database (SADB).

SA Negotiation

IKE implements portions of the Internet Security Association Key Management Protocol (ISAKMP) and the Oakley key exchange protocol. ISAKMP provides a framework for SA negotiation and the Oakley protocol governs the exchange of key material between network peers.

When IKE runs on two network peers, either instance can initiate negotiation. The initiator consults its security policy database (SPD) for the protection suites it can offer, then sends one or more proposals to the responder.

A protection suite consists of one or more IPsec protocols and their attributes. Protocol attributes include cryptographic algorithms and their keying material. On receiving a request for a protection suite, the initiator suggests a suitable protection suite to the responder by sending it a proposal. For example, a proposal could specify “ESP with 3DES and MD5, followed by AH with SHA-1, applied successively on the same packet”.

The responder consults its SPD and proposes a protection suite that it can accept. Once the peers agree on a protection suite, they negotiate to establish the SA to be used by IPsec. Negotiation occurs in two phases, as described below. Finally, each peer adds the negotiated SA to its SADB.

If the peers cannot agree on a protection suite, they handle IP traffic according to their default policies.

Phase 1

In Phase 1, two IKE tasks establish a secure channel between themselves, fully authenticating each other and establishing key material for encrypting and authenticating Phase 2 communications.

Phase 1 (Main Mode) has three steps:

1. Negotiation of an SA for secure exchange of IKE messages.
2. Exchange of Diffie-Hellman public key values and nonces. This part is optional and is part of the PFS process.
3. Authentication by exchange of preshared keys or X.509 certificates.

IKE uses the established secret from step 2 (the DH Exchange) to derive the keys for IPsec SAs. The same secret is used to derive keys when rekeying IPsec SAs, unless Perfect Forward Secrecy (PFS) is enabled. If PFS is enabled, IKE performs a new DH Exchange (step 2) when rekeying the IPsec SAs.

Phase 2

Phase 2 performs quick SA setup once the slower Phase 1 is complete. The two IKE tasks create a pair of SAs for the peers that want to communicate. These peers may be the hosts on which the IKE tasks run, a host and a network behind a firewall, or two networks behind their respective firewalls. Phase 2 specifies the identities of the communicating peers and the SA parameters (algorithms to use, encapsulation mode, lifetime), then exchanges the authentication and encryption keys to be used by IPsec until the next Phase 2 negotiation takes place. Multiple Phase 2 setups can occur over the same Phase 1 channel.

Phase 2 (Quick Mode) has three steps:

1. Negotiation of the SA to be used by IPsec.
2. Exchange of Diffie-Hellman public key values and nonces.
3. Transfer of the SA, keys, and Security Parameters Index (SPI) to IPsec.

Parent topic: How IKE Works on page 4

1.2. Authentication Using a Preshared Key

Network peers can establish trust in each other through the use of a preshared key (also called a preshared secret).

At the start of negotiation, the peers compare their stored key values. If the keys do not match, authentication and negotiation fail.

Specify the key value by means of the shared_key entry in the authentication section of the IKE configuration file.

Parent topic: How IKE Works on page 4

1.3. Authentication Using Certificates

Network peers can establish trust in each other through the exchange of X.509 certificates.

A certificate binds a peer's public key to the peer's identity. In order to be trusted, the certificate must itself be signed by a trusted Certificate Authority (CA) or by a member of a chain of trust that ends at such an authority.

A public key certificate contains the following information:

- the identity of the key owner, which can be a full X.500 Distinguished Name (DN) or certain parts of the DN, as determined by the CA
- the validity period of the certificate, consisting of a not-before date and a not-after date
- the key owner's public key
- the identity of the issuer CA, which can be a full X.500 DN or certain parts of the DN, as determined by the issuer
- the issuer's digital signature, consisting of a hashed digest of the certificate's contents, encrypted with the issuer's private key
- administrative information used by the CA, such as a serial number
- other application-specific information and extensions

WNDRVR

For background information about X.509 certificates, see RFC 2459: *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*.

In Phase 1, before commencing negotiation, the peers exchange their RSA public keys and public key certificates. Each peer validates the other's certificates; if a certificate cannot be validated, the authentication fails. If the validation succeeds, each peer signs its negotiation messages using its private key. Each peer uses the other's public key to authenticate the signature and decrypt the messages.

Certificates and private keys must be PEM-encoded (Privacy Enhanced Mail) and stored in accessible file systems on the target machines. Specify the locations of a peer's private key file and public key certificate file by means of the private_key and certificate entries in the **authentication** section of the IKE configuration file.

If the private key file is password-protected, specify the password by means of the private_key_password entry.

Parent topic: How IKE Works on page 4

1.4. Perfect Forward Secrecy

Perfect Forward Secrecy protects against the compromise of a private or current session key putting earlier or subsequent keys at risk.

RFC 2409: *The Internet Key Exchange (IKE)*, states the principle of PFS: the key used to protect a transmission of data must not be used to derive any additional keys, and if the key used to protect a transmission of data was derived from some other keying material, that material must not be used to derive any more keys.

If PFS is enabled, Phase 2 performs a new Diffie-Hellman computation of key material. This operation significantly slows down the exchange but ensures that the Phase 2 traffic will still be protected even if an attacker breaks the Phase 1 exchange. If PFS is not enabled, Phase 2 uses the key material generated in Phase 1 to establish keys for the transforms to be used by IPsec.

Normally, PFS is not enabled. The key material used by the IPsec SAs is derived from the key material of the Phase 1 exchange, allowing Phase 2 to complete quickly without public key computations.

To enable PFS in Wind River IKE, code the following entries in the IKE configuration file:

```
proposal {
                dh_group modp1024;
                encryption_algorithm 3des;
                integrity_algorithm sha1;
}
```

To disable PFS, code the following entries.

```
proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
}
```

Parent topic: How IKE Works on page 4

1.5. Flows

In Wind River IKE, flows act as unique identifiers for sets of IPsec SA negotiation parameters (in the IKE configuration file) and for individual SAs (in the SADB).

A flow is a 5-tuple set of packet selectors consisting of a source IP address, a destination IP address, a source port, a destination port, and an IP protocol identifier.

On receiving a request for an IPsec SA, Wind River IKE matches the flow contained in the request against the available IPsec SA configurations. The first match found provides the parameters for negotiation. When negotiation completes, Wind River IKE adds the new SA, identified by the flow, to the SADB.

Because incoming IKE Phase 1 traffic has to bypass IPsec processing, Wind River IKE automatically adds a bypass flow for the address and port of its own host.

Note that use flows are created with a default intermediate priority of -50.

Parent topic: How IKE Works on page 4

2. IKE Dynamic Address Mode

Wind River IKE supports dynamic address mode.

In this mode, a dynamic IP address is temporarily assigned from a pool of addresses. At the end of a session, the address is returned to the pool of available addresses.

Dynamic addressing is determined by the IKE initiator, which requests a dynamic address from the responder.

Dynamic addressing is typically used by VPN clients when tunneling into a protected network. A VPN client makes a connection to an ISP. It then negotiates an IKE SA with the VPN gateway. Once the IKE SA is negotiated and authenticated, the gateway initiates dynamic address mode with the VPN client. Once the client has a dynamic address, it can load the IPsec SA from the gateway.

In IKEv1, the dynamic address is negotiated during the transactional exchange. You configure IKEv1 dynamic addressing using the IPIKE_DYNAMIC_ADDRESS_MODE parameter. Use the IPIKE_DYNAMIC_ADDRESS_POOL_DNS4 parameter to define a DNS server address that can supply a dynamic address to the responder from a pool of addresses. The IPIKE_DYNAMIC_ADDRESS_POOL_DNS4 parameter specifies the name server and is handed out to the client along with the dynamic IP address. The parameter IPIKE_DYNAMIC_ADDRESS_POOL_NETWORK4 defines a network range of addresses and subnet mask to supply.

For more IKEv1 dynamic address configuration information, see the parameter descriptions in the online Wind River IKE Component Reference, accessible through the Workbench online Help.

You can modify the reference file **ipike_dynamic_ip.c** to extend support to IPv6 addresses. The file is located at the following location:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-x.x.x.x/ike/src/ipike_dynamic_ip.c**

In IKEv2, the dynamic address is negotiated for IPsec SA during the SA INIT exchange.

You configure IKEv2 dynamic addressing using the dynamic_address IPSEC_SA parameter in the IKE configuration file.

For IKEv2 dynamic address configuration information, see IPsec SA Parameters on page 29.

Dynamic Address Mode Example Use Cases on page 8 The IPIKE_DYNAMIC_ADDRESS_MODE setting is used primarily because, with IKEv1, the responder is not allowed to narrow the initiator's traffic (flow) selectors, so the selectors proposed by the initiator must be accepted by the responder.

2.1. Dynamic Address Mode Example Use Cases

The IPIKE_DYNAMIC_ADDRESS_MODE setting is used primarily because, with IKEv1, the responder is not allowed to narrow the initiator's traffic (flow) selectors, so the selectors proposed by the initiator must be accepted by the responder.

This becomes tricky when dynamic IP is used and the initiator might not know what address or subnet it will receive.

Depending on the responder's policy the initiator may be forced to adapt the selector it proposes. For example, when a responder that is configured to deliver an IP address on the 10.1.111.0/24 network and is also configured with the selectors (0.0.0.0/0-10.1.111.0/0), an initiator must then be configured with a matching flow selector, but since the initiator might not know beforehand the correct selectors it is better to use the dynamic IP information it received. In most cases, it would make the most sense for the initiator to propose the received IP address as the source address

```
IPIKE_DYNAMIC_ADDRESS_MODE=1
```

or

```
ipike.dynamic_address.mode=1
```

in its selectors, that is, (10.1.111.2/32-0.0.0.0/0).

An example use case for setting the IPIKE_DYNAMIC_ADDRESS_MODE parameter to 2 or the sysvar, *ipike.dynamic_address.mode* =2, would be a situation in which the initiator is acting as gateway for more hosts on the 10.1.111.0/24 network. The resulting IPsec flow would allow any host on the 10.1.111.0/24 network to use the IPsec tunnel instead of a single host.

Note that IKEv2 solves this issue by allowing the responder to narrow the selectors that are proposed by the initiator. The initiator could be configured with selectors (0.0.0.0/0-0.0.0.0/0) and the responder could for example narrow the selectors to (10.1.111.2-10.1.111.0/24) and send them back to the initiator.

Parent topic: IKE Dynamic Address Mode on page 8

3. PF_KEY v2 Key Management Interface

Learn about the PF_KEY v2 key management interface, based upon RFC 2367: PF_KEY Key Management API, Version 2.

For detailed information, see this and other RFCs listed in RFCs on page 2.

PF_KEY v2 is a generic key management API definition. Its purpose is to provide an OS-independent interface between key management applications, such as Wind River IKE, and OS-specific key engines, such as the SADB of Wind River IPsec. PF_KEY v2 uses a defined set of messages to insert SA information into and retrieve information from the key engine.

When IPsec processes an IP packet according to policy, it may request an SA from the key engine. If the key engine has no suitable SA, but a key management application has previously indicated, by means of the **SADB_REGISTER** message, that such SAs can be obtained, then the key engine requests the creation of a new SA by means of an **SADB_ACQUIRE** message. The key management application adds the new SA to the key engine for future use.

There are two basic ways to add a new SA to the key engine. The simplest approach is for the key management application to send a single **SADB_ADD** message, containing all of the SA information, to the key engine. The second approach is for the application to request a Security Parameter Index (SPI) by means of the **SADB_GETSPI** message, and then send an **SADB_UPDATE** message with the complete SA data.

The use of PF_KEY v2 enables Wind River IKE to interoperate with any PF_KEY v2-compliant implementation of IPsec. Wind River IKE implements the PF_KEY v2 API definition internally to interface with the SADBs of Wind River IPsec and Linux IPsec. For descriptions of the internal functions, see the *Wind River IPSec and IKE API reference*. If you use another vendor's IPsec, you may have to implement these functions differently.

4. UDP Encapsulation and NAT-Traversal

Wind River IKE implements the recommendations of RFC 3947: Negotiation of NAT-Traversal in the IKE, enabling it to detect network address translation (NAT) devices situated between IPsec hosts.

About This Task

IKE negotiates the use of UDP encapsulation for NAT-Traversal (NAT-T), enabling IPsec packets to traverse (pass through) NAT devices.

Procedure

Do one of the following to enable NAT detection:

Option	Description
Build time	set the IPIKE_NAT_PORT configuration parameter to **4500**.
Run time	set the *ipike.port_nat sysvar* to **4500**.

Results

Wind River IKE then automatically enables UDP encapsulation when it detects a NAT device, changing the IKE negotiation port from 500 to 4500 and the encapsulation mode for Phase 2 negotiation from transport or tunnel to **udp_transport** or **udp_tunnel**, respectively.

5. Dead Peer Detection

Learn about Wind River support for dead peer detection, based upon RFC 3706: A Traffic-Based Method of Detecting Dead Internet Key Exchange (IKE) Peers.

When two peers communicate using IPsec and IKE, a situation may arise in which connectivity between the two goes down unexpectedly; for example, because of routing problems or if one of the hosts reboots. In such cases, there is often no way for IPsec and IKE to detect the loss of connectivity. SAs may remain in existence (until their lifetimes naturally expire), resulting in a “black hole” situation where packets are tunneled to oblivion. It is desirable to recognize black holes as soon as possible so that a host can quickly fail over to a different peer. Likewise, detecting black holes enables the recovery of lost resources.

This problem of detecting a dead IKE peer has been addressed by proposals that require sending periodic HELLO/ACK messages to prove liveliness. These schemes tend to be unidirectional (a HELLO only) or bidirectional (a HELLO/ACK pair). For the purpose of this document, the term heartbeat refers to a unidirectional message to prove liveliness. Likewise, the term keepalive refers to a bidirectional message.

The problem with current heartbeat and keepalive proposals is their reliance upon the sending of messages at regular intervals. In practice, this scheme requires a timer to service the message intervals. Because rapid detection is desired, the messages have to be sent frequently, causing considerable message processing overhead. In implementations where a large number of simultaneous IKE sessions is required, these regular heartbeat and keepalives are unfeasible.

Wind River IKE implements the scheme proposed in RFC 3706 for determining when to query a peer for liveness using DPD messages. Wind River IKE reacts to an absence of incoming traffic (both IKE and IPsec) instead of just sending DPD heartbeats at regular intervals. The liveness parameter in the **ike_sa** configuration file is a measurement of how long a time with an absence of

incoming traffic the ike_sa accepts before sending a DPD message. Both IKEv1 and IKEv2 use the same mechanism for triggering a transmission of heartbeat messages.

For more information, see Dead Peer Detection (DPD) on page 53.

6. IKEv2

IKEv2, the second version of the IKE protocol, uses the same techniques as IKEv1, but improves upon IKEv1 in several ways.

- It offers simpler message exchange.
- It relies on fewer cryptographic mechanisms.
- It has greater reliability through state management.
- It provides better resistance to denial-of-service attacks.

To enable the use of IKEv2 in Wind River IKE, define the IPIKE_USE_VERSION2 parameter in the IKE initialization code. Then set the exchange_version entry in the **ike_sa** section of the IKE configuration file to specify the IKE version to be used by the initiator. The responder will accept both v1 and v2.

Authentication Using EAP on page 11 The Extensible Authentication Protocol (EAP) is available in IKEv2.
Online Certificate Status Protocol (OCSP) on page 11 Wind River IKE supports Online Certificate Status Protocol (OCSP) extensions to IKEv2, as defined in RFC 4806.
RADIUS Support on page 12 When EAP is enabled in IKEv2, Wind River IKE also provides the option of using RADIUS for authentication, authorization, and accounting in network access or IP mobility applications.
Mobility and Multihoming on page 12 Wind River supports IKEv2 mobility and multihoming, based upon RFC 4555: IKEv2 Mobility and Multihoming Protocol (MOBIKE).
Anti-replay Window on page 12ESN (Extended Sequence Number) provides anti-replay support for high-speed IPsec implementations.
Renegotiation of Diffie-Hellman Value in Proposals on page 13 Wind River IKE supports renegotiation of the Diffie-Hellman group value of the proposal both as responder and as initiator. This is supported only in IKEv2.
ECN Support on page 14 Wind River IKE provides Explicit Congestion Notification (ECN) support.

6.1. Authentication Using EAP

The Extensible Authentication Protocol (EAP) is available in IKEv2.

EAP is a universal authentication framework used in point-to-point connections. EAP is not an authentication method; rather, the EAP framework provides the means to negotiate the desired authentication method. EAP-IKEv2 is one such method.

EAP-IKEv2 provides mutual authentication and session key establishment between an EAP peer and an EAP server. It supports authentication based on the following types of credentials:

- asymmetric key pairs, where the public key is in a certificate
- passwords
- symmetric keys

To specify the use of EAP-IKEv2 authentication in Wind River IKE, define the IPIKE_USE_EAP parameter in the IKE initialization code. For more information about EAP, see the *Wind River EAP Programmer's Guide*.

Parent topic: IKEv2 on page 11

6.2. Online Certificate Status Protocol (OCSP)

Wind River IKE supports Online Certificate Status Protocol (OCSP) extensions to IKEv2, as defined in RFC 4806.

OCSP is a mechanism for IKEv2 that enables two communicating peers to exchange certificate revocation status information in-band.

The typical way of determining certificate revocation status with IKEv2 is through the use of in-band Certificate Revocation Lists (CRLs). However, the in-band use of CRLs is prohibitive due to the unbounded size of CRLs. The size of an OCSP response is small and bounded, and therefore more suitable for in-band use.

For information about configuring VxWorks with OCSP, see OCSP Configuration on page 36.

Parent topic: IKEv2 on page 11

6.3. RADIUS Support

When EAP is enabled in IKEv2, Wind River IKE also provides the option of using RADIUS for authentication, authorization, and accounting in network access or IP mobility applications.

To specify the use of RADIUS authentication, define the IPIKE_USE_RADIUS parameter in the IKE initialization code.

For more information about RADIUS, see the *Wind River RADIUS and Diameter for VxWorks 7 Programmer's Guide*.

Parent topic: IKEv2 on page 11

6.4. Mobility and Multihoming

Wind River supports IKEv2 mobility and multihoming, based upon RFC 4555: IKEv2 Mobility and Multihoming Protocol (MOBIKE).

The mobility and multihoming capabilities of Wind River IKE are intended to be used with Wind River Mobile IPv6, which is an optional feature of the Wind River Network Stack.

IKEv2 creates IKE SAs and tunnel mode IPsec SAs using the IP addresses specified when the IKE SA is established. These addresses are then used as the outer (tunnel header) addresses for tunnel mode IPsec packets. In the base IKEv2 protocol, it is not possible to change these addresses after the IKE SA has been created.

However, there are scenarios where these IP addresses might change. One example is mobility: a host might change its point of network attachment and receive a new IP address. Another example is a multihoming: a host might need to change to a different interface if, for example, the current interface stops working.

Although addresses can be changed by creating new IKE and IPsec SAs, this may not be the optimal solution. Creating a new IKE SA may require user interaction for authentication, such as entering a code from a token card. Creating new SAs often involves expensive calculations and possibly a large number of round-trips.

The MOBIKE protocol provides a mechanism for updating the IP addresses of existing IKE and IPsec SAs. The main scenario for employing MOBIKE is enabling a remote access VPN user to move from one address to another without having to re-establish SAs with the VPN gateway.

To enable MOBIKE in Wind River IKE, set the *ipike.mobike* sysvar to 1.

Parent topic: IKEv2 on page 11

6.5. Anti-replay Window

ESN (Extended Sequence Number) provides anti-replay support for high-speed IPsec implementations.

There is a dependency on IKEv2 for automatically configured SAs to use ESN. If the SAs are manually configured, the keyadm command must be used. The default is to use ESN unless it is turned off. Setting the replay window to 0 disables anti-replay protection.

ESN is configured in IKE using the use_esn parameter in the **proposal** section of the IKE configuration file. See Proposal Parameters Reference on page 39.

The IPsec keyadm command configures ESN usage to specify window sizes or disable replay protection. The option -replay64 is used during the creation of an AH or ESP SA to enable or disable 64-bit replay protection. Window sizes of 8, 16, 24, n*8, to a maximum size of 1024, are supported. Specifying a window size of 0 disables replay protection. For more information see the *VxWorks 7 IPsec Programmer's Guide*

Parent topic: IKEv2 on page 11

6.6. Renegotiation of Diffie-Hellman Value in Proposals

Wind River IKE supports renegotiation of the Diffie-Hellman group value of the proposal both as responder and as initiator. This is supported only in IKEv2.

This functionality works as follows: The IKE initiator selects and uses the first DH group specified in the **ike_sa** proposal configuration, but sends all configured DH groups in its proposal to the responder.

Next, one of the following can happen:

1. The selected DH group is accepted by the IKE responder (indicating that it is part of the IKE responder's configuration).
2. None of the DH groups matches the IKE responder configuration, and the responder fails the negotiation with the message “No proposal chosen.”
3. The selected (and used) DH group is not accepted but the initiator's proposal contains another DH group that the responder accepts. The IKE responder then replies with a response containing **INVALID_KE_PAYLOAD** and an accepted DH group. The IKE initiator can then process the response and re-send its initial request using the correct DH group.

Example

The following example shows how initiator and responders could be configured for this feature.

```
-------- Initiator------------
proposal {
        dh_group 1,2;
        hash_algorithm sha1;
        encryption_algorithm 3des;
        integrity_algorithm sha1;
}
------------------------------

--------- Responder ----------
proposal {
        dh_group 2,5;
        hash_algorithm sha1;
        encryption_algorithm 3des;
        integrity_algorithm sha1;
}
------------------------------
```

Parent topic: IKEv2 on page 11

6.7. ECN Support

Wind River IKE provides Explicit Congestion Notification (ECN) support.

ECN is a mechanism for indicating the occurrence of congestion between tunnel endpoints.

The -ecn option of the IPsec keyadm add/update commands controls the ECN setting. Enabling ECN means that ECN bits are handled as specified in RFC4301. See the *VxWorks 7 IPsec Programmer's Guide* for more information about keyadm.

For IKE negotiated IPsec SAs (both IKEv1 and IKEv2), ECN is enabled by default. A configuration option, use_ecn, in the **ipsec_sa** section of the IKE configuration file specifies whether ECN is on or off.

See IKE SA Parameters Reference on page 26 for more information.

Parent topic: IKEv2 on page 11

7. Certificate Authentication using Hash and URL

Wind River IKE provides Hash and URL certificate authentication support.

This support provides a means of using certificates out-of-band by utilizing a hash and a URL location for certificate retrieval.

When using certificates as the authentication method, the IKE packet could get big and then IP fragmentation would be required. With Hash and URL support, this problem is avoided, since the certificate is not sent in-band in the IKE negotiation, but instead is sent using a separate HTTP request over TCP.

This works as follows:

An IKE peer will instead of sending the certificate send a 20bytes SHA1 hash of the certificate plus one URL pointing out the certificate. The receiver will then fetch the certificate over HTTP and then continue the authentication process as if the certificate was received in-band of the IKE negotiation.

Hash and URL certificate authentication support can be configured at build time using the following components:

- **IPIKE_HTTP_REQUEST_TIMEOUT**
- **IPIKE_HTTP_RESPONSE_BUF_SIZE**
- **IPIKE_USE_HASH_AND_URL**

For more details, see Configuring and Building Wind River IKE on page 23.

Hash and URL certificate authentication support can also be configured at runtime using the following sysvars:

- *ipike.http_request_timeout*
- *ipike.http_response_buf_size*
- *ipike.use_hash_and_url*

For more details, see Programming the Initialization Code on page 45.

A configuration parameter, certificate_uri, in the **authentication** section in the **ike_sa** section of the IKE configuration file, specifies the base location of the certificate that together with the SHA1 hash forms the complete URI pointing out the certificate. The following examples show acceptable formats for the certificate_uri specification:

```
certificate_uri http://windriver.com/certificates/;
```

or

```
certificate_uri http://172.16.0.11/user/certs/;
```

or

```
certificate_uri http://2001:1::5/;
```

The certificate file is specified as usually, and a 20 byte hash is calculated of the raw DER encoded certificate and appended to the certificate_uri by the IKE daemon. For example, the full URI received by the other end could be:

http://webserver.com/certificates/8ba8eaa17653f6b434c64fa6049ca5862363c111;

Of course, for this to work successfully, there must be a certificate named **8ba8eaa17653f6b434c64fa6049ca5862363c111** on the web server.

The certificate placed on the web server must be in the raw DER encoded format.

8. CRL Support

Wind River IKE verifies the validity of a given certificate against a Certificate revocation list, issued by the appropriate Certificate Authority.

The location of the CRL store is specified in the **authentication** section of the IKE SA.

This feature is available for both IKEv1 and IKEv2.

For more information, see CRL Support on page 34.

Note: The crl_store entry is only valid when certificate-based authentication is used.

9. Fetching CRLs Remotely Using HTTP

Wind River IKE provides support for fetching a Certificate Revocation List (CRL) from a server using HTTP.

Wind River IKE provides support for fetching a Certificate Revocation List (CRL) from a server using HTTP. This functionality is supported in both IKEv1 and IKEv2.

Fetching a CRL using HTTP support can be configured at build time using the following components:

- **IPIKE_HTTP_REQUEST_TIMEOUT**
- **IPIKE_HTTP_RESPONSE_BUF_SIZE**

For more details, see Configuring and Building Wind River IKE on page 23.

Fetching a CRL using HTTP support can also be configured at runtime using the following sysvars:

- *ipike.http_request_timeout*
- *ipike.http_response_buf_size*

For more details, see Programming the Initialization Code on page 45.

The crl_store parameter in the **authentication** section of the IKE configuration file can contain a URI pointing out a CRL or a directory on local storage. For example:

```
crl_store http://webserver.com/cert/revoke.crl;
```

If the CRL usage has been specified and the CRL cannot be fetched, the IKE negotiation will fail due to authentication failure.

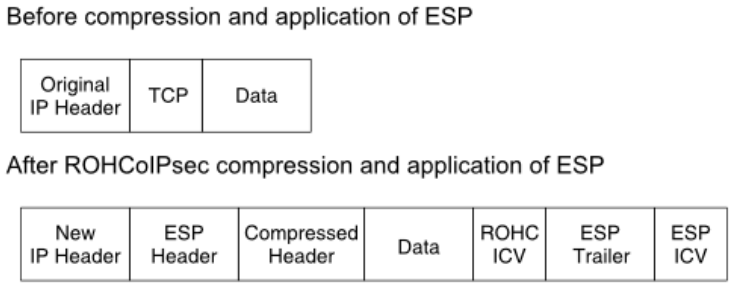
10. Robust Header Compression over IPsec

Wind River IKE supports signaling of robust header compression (ROHC) parameters (RFC-5857).

This support is required in order to performing robust header compression over IPsec (ROHCoIPsec) (RFC-5857). Header compression in IPsec can only be applied in tunnel mode SAs.

The headers of the tunneled traffic are compressed. ROHCoIPsec can also be combined with ordinary ROHC to compress outer headers of an ESP/AH packet.

Figure 4. ROHC as Applied on a Packet Processed by a Tunnel Mode ESP SA



For an example illustrating ROHCoIPsec, see ROHCoIPsec-Processed Packet Example on page 88.

11. Cryptographic Support

Build-time configuration of Wind River IKE includes the selection of a cryptographic provider module.

At run time, configuration of the IKE task includes the selection of cryptographic algorithms to be used in proposals for SA negotiation. For an algorithm to be usable, both Wind River IKE and the provider module must have the necessary capabilities.

Cryptographic Provider Modules

Wind River IKE obtains cryptographic services by passing requests to a cryptographic provider module, which handles communications between IKE and the required cryptographic algorithms. The algorithms themselves can be implemented in either software or hardware.

The Wind River Cryptography Libraries supply cryptographic algorithm support to VxWorks applications. Wind River products such as Wind River IKE call the functions as needed. In addition, other applications can call the Cryptography Libraries functions using the standard OpenSSL API.

If desired, you can add support for new algorithms. To add an algorithm, code its name, identifier, key length, and other required information in the mapping table in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ike/src/ipike_map.c**. Wind River IKE can then use a hardware or software implementation of the algorithm.

Supported Cryptographic Algorithms Reference on page 17

11.1. Supported Cryptographic Algorithms Reference

Symmetric Cipher Algorithm Support in Wind River IKE

The following table lists the Symmetric Cipher algorithms supported in Wind River IKE.

Symmetric Cipher Algorithm Type	Standards	Wind River IKE Support
AES128-CBC	NIST FIPS 197	Phase 1 and 2
AES192-CBC	NIST FIPS 197	Phase 1 and 2
AES256-CBC	NIST FIPS 197	Phase 1 and 2
DES-56-CBC	NIST FIPS 46-3	Phase 1 and 2
3DES-56-CBC (DES-56-CBC)	NIST FIPS 46-3	Phase 1 and 2
3DES-168-CBC	NIST FIPS 46-3	Phase 1 and 2
Blowfish-128		Phase 1 and 2
CAST-128	NIST FIPS 46-3	Phase 1 and 2

Hash and Message Digest Support in Wind River IKE

The following table lists the Hash and Message Digest algorithms supported in Wind River IKE.

Hash and Message Digest Type	Standards	Hash sum size
MD5-128	RFC 1321	128 bits
SHA-256	NIST FIPS 180-4	
SHA-384	NIST FIPS 180-4	
SHA-512	NIST FIPS 180-4	

Message Authentication Code Support in Wind River IKE

The following table lists the Message Authentication Code algorithms supported in Wind River IKE.

Message Authentication Code Type	Standards	Wind River IKE
AES-XCBC-MAC-96 (128 bit support)	NIST FIPS 197	Phase 1
AES-XCBC-MAC-128	NIST FIPS 197	Phase 1
HMAC-SHA1	RFC 2104	
HMAC-MD5	RFC 2104	Phase 1
HMAC-SHA1-256	RFC 4868	Phase 1
HMAC-SHA1-384	RFC 4868	Phase 1
HMAC-SHA1-512	RFC 4868	Phase 1

Public Key Encryption and Key Exchange in Wind River IKE

The following table lists the Public Key Encryption and Key Exchange algorithms supported in Wind River IKE.

Public Key Encryption and Key Exchange Type	Standards
Diffie-Hellman: **group 1 (768 bits)** **group 2 (1024 bits)** **group 5 (1536 bits)** **group 14 (2048 bits)** **group 15 (3072 bits)** **group 16 (4096 bits)** **group 17 (6144 bits)** **group 18 (8192 bits)**	RFC 2631 and RFC 3526
RSA (1024-4096 bits)	
PKCS (PKCS7, PKCS8, PKCS12)	RFC 3447

Psuedo-random Functions

The following table lists the psuedo-random functions supported in Wind River IKE.

Function	Standards
AES-XCBC-PRF-128	RFC 4434
PRF-SHA1	RFC 2104

Function	Standards
PRF-MD5	RFC 2104
PRF-HMAC-SHA-256	RFC 4868
PRF-HMAC-SHA-384	RFC 4868
PRF-HMAC-SHA-512	RFC 4868

Parent topic: Cryptographic Support on page 16

12. Cryptographic Suites for IPsec for VPN Applications

Wind River IPsec complies with RFC 4308: Cryptographic Suites for IPsec.

This RFC specifies suites of attributes and algorithms that can be used to administer IPsec in manual keying mode with either IKEv1 or IKEv2. Manual keying mode (or manual configuration) means using the keyadm command with IPsec; automatic configuration means using IKE for negotiation.

Wind River IPsec implements the VPN-A and VPN-B suites as described in RFC 4308. These cryptographic suites are for use in IPsec virtual private networks.

For configuration information and examples, see Configuring Cryptographic Suites for VPNs on page 51.

13. IKE Multipoint VPN Support

Wind River IKE Multipoint VPN support enables a local ID or remote ID to be used for the IPsec/IKE SA selection process in addition to static IP addresses.

This means that you can have multiple configurations for the same remote IP address space (usually 0.0.0.0/0) and distinguish between them based on the ID parameter.

Wind River IKE by default supports choosing the IPsec/IKE SA configuration based on IP addresses. This means we typically use the IP of the remote system to determine the configuration to be used. This is an acceptable way of handling it for many static use cases (for example, tunnels connecting two office networks), but it is insufficient to support dynamic use cases, such as remote users who travel and log in from different addresses at different times and locations. To provide IPsec/IKE SA configuration support for such cases, Wind River IKE provides IKE Multipoint VPN support.

Using Wind River IKE Multipoint VPN support it is possible to have a responder configuration, for example, that looks similar to this:

```
ike_sa address 0.0.0.0/0 {
ike_id tunnel1
remote_identifier user_fqdn t1@wr.com;
...
}

ike_sa address 0.0.0.0/0 {
ike_id tunnel2;
remote_identifier user_fqdn t2@wr.com;
```

```
...
}
```

An initiator with ID "t1@wr.com" would match the first **ike_sa** configuration and an initiator with ID "t2@wr.com" the second one.

There is one limitation that should kept in mind. In both IKEv1 with Main Mode and in IKEv2, the initiator's ID is not sent before the second message exchange, and the IKE SA proposal is negotiated in the first exchange. This forces all **ike_sa** configurations (that cannot be separated by remote address) to use the same proposal parameters.

However, in IKEv1 Aggressive Mode, all parameters from the second exchange (including the initiator's ID) are pushed into the first message exchange and it is then possible to differentiate connections from different IDs already from the first message. Since Aggressive Mode is less secure than Main Mode, and its primary advantage over Main Mode is only that it is a bit quicker, its use is not recommended.

For example configurations that demonstrate the use of this feature, see Examples: Configurations for IP Multipoint VPN Access on page 73.

14. Custom APIs

Wind River IKE includes certain APIs that you can customize for special purposes.

For definitions of these APIs, see the *Wind River IPSec and IKE API reference*.

Policy API

The default implementation of the Policy API selects transforms and proposals for SA negotiation by consulting the **proposal** sections of the IKE configuration file. If, however, you choose to store transform and proposal information in another place, it will be necessary to implement this API differently.

See the **ipike_policy.c** and **ipike_policy.h** files for reference information when customizing the Policy API. These files provide detailed explanations of the functions that can be extended. The files are located in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ike/src**.

PF_KEY v2 API

The default implementation of the PF_KEY v2 key management API interfaces with the SADB of Wind River IPsec or Linux IPsec. If you use another vendor's IPsec, you may have to provide your own implementation of the entire API. If you want to write a custom logging facility for SA events, you must implement the ipike_pfkeyv2_process_sa_event( ) function.

PFKEY v2 is a standard API and IKE usage examples can be seen in the files **ipike_pfkeyv2.c** and **ipike_pfkeyv2.h**. The files are located in *installDir* **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ike/src**.

15. ESP and AH Support in a Single Proposal

Wind River IKE (IKEv1) supports accepting both ESP and AH negotiation in the same proposal for a given IPsec flow configuration.

This is required for interoperability with Windows Vista and Windows Server equivalents which allow such policy specification. For more information, see Configuration Files for Scenario 1 on page 82.

16. Logging and Statistics

Wind River IKE logs events, such as SA timeouts, to the syslog task.

The amount of information recorded depends on the debug level. You can set the debug level by means of the IPIKE_SYSLOG_PRIORITY parameter macro in the IPsec initialization code. Alternatively, you can use the syslog shell command. Messages output to syslog are printed by default on a debug port, but the logging function can also write the log entries to a file or a UDP socket. The amount of information recorded depends on the debug level. You can set the debug level using the syslog shell command. For example, the command:

```
syslog priority ipike Emerg
```

would log all IKE related Emergency level messages to syslog.

Syslog Synopsis

- syslog echo prio message
- syslog list
- syslog priority facility prio
- syslog log file *logfile*

Where facility is **ipike** and **prio** is the debug priority level of the specified message. Options include:

- Emerg
- Crit
- Error
- Warning
- Notice
- Info
- Debug
- Debug2

The following table lists the syslog message priority levels.

Table 1.

Level Name	No	Workbench Configuration Parameter	Priority
EMERGENCY	0	IPCOM_LOG_EMERG	Highest priority.
CRITICAL	1	IPCOM_LOG_CRIT	
ERROR	2	IPCOM_LOG_ERR	Default log output level at run time, as defined by IPCOM_SYSLOGD_DEFAULT_PRIORITY.
WARNING	3	IPCOM_LOG_WARNING	This level and higher compiled in for normal builds.
NOTICE	4	IPCOM_LOG_NOTICE	

Level Name	No	Workbench Configuration Parameter	Priority
INFO	5	IPCOM_LOG_INFO	
DEBUG	6	IPCOM_LOG_DEBUG	Lowest priority. This level and higher compiled in for debug builds.
DEBUG 2	7	IPCOM_LOG_DEBUG2	Default level when debug flag is used.

For complete information on syslog, see the Wind River Network Stack documentation set.

3. CONFIGURING AND BUILDING WIND RIVER IKE

To configure a VxWorks image project with support for Wind River IKE, build a VSB project with the required components.

About This Task

Then build a VIP based on that VSB, and include the appropriate components and set values for parameters.

For details on configuring a VxWorks image project (VIP), see the configuration sections in the *VxWorks 7 Configuration and Build Guide*. For more information on the component parameters and their default values, see the online *Wind River IKE Component Reference*.

For information on further configuring Wind River IKE at run time, see *VxWorks 7 IKE Programmer's Guide:* *Run-time Configuration on page 24*.

Procedure

1. Configure a VxWorks Source Build (VSB).

   In your VSB project, enable the IPNET_IKE option

2. Create and configure a VxWorks Image Project (VIP).
   1. Create a VIP based on the VSB project.
   2. Add the INCLUDE_INTERNET_KEY_EXCHANGE component to the VIP.
3. Set the INCLUDE_INTERNET_KEY_EXCHANGE configuration parameters.

   Set values for the static configuration parameters

4. Build the VIP project.

4. RUN-TIME CONFIGURATION

1. Introduction

Certain configuration operations can be performed while Wind River IKE is running.

These run-time operations are of two types: IKE task configuration and SA administration. Run-time configuration operations can be performed in several ways. This chapter describes the following methods:

- using a configuration file
- programming the initialization code
- running shell commands (see Running Shell Commands on page 96.)

  In addition, you can write an application program to configure Wind River IKE while it is running. Wind River IKE includes a configuration API that sets many of the same parameters as the configuration file. For reference documentation on these functions, see the reference entries for the functions.

2. IKE Configuration File

Wind River IKE initializes in a default configuration when the target boots.

You can change the default configuration by supplying a configuration file. At startup, or when restarted by the ipd start ipike shell command, the IKE task reads configuration parameters from a file specified by the Workbench parameters IPIKE_CONFIG_FILE_NAME and IPIKE_CONFIG_FILE_PATH.

The configuration file is an ASCII text file containing elements of two types: sections and entries. A section consists of a name and an optional value, followed by a bracketed list of other sections or entries. An entry consists of a name and a value, followed by a semicolon. Names and values are case-insensitive. Comment lines, starting with the **#** character, can be placed anywhere in the file. The file format is shown symbolically below:

```
<element> ::= <section> | <entry>

<section> ::= name [<value>] {
    array of <element>
}
<entry> ::= <name> <value>;
```

There are three top-level sections: **general**, **ike_sa**, and **ipsec_sa**. There can be multiple **ike_sa** and **ipsec_sa** sections, but only one **general** section.

If the configuration file name and path are not specified, and the parameter IPIKE_CREATE_DEFAULT_CONFIG_FILE is set to 1, the IKE task writes its configuration to the file **/ram/ike.cfg**. (The default file configures the host as a responder that authenticates using the preshared key this_should_be_secret.)

The structures used in the IKE configuration file are defined in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ike/h/ ipike_configuration_api.h**

For examples of creating, deleting, and checking for the existence of IKE and IPsec SAs, see *VxWorks 7 IKE Programmer's Guide:* *IKE Configuration API Examples on page 65*.

General Configuration Parameters Reference on page 25 The **general** section of the IKE configuration file contains parameters that configure the IKE task.
IKE SA Parameters Reference on page 26 The **ike_sa** section contains parameters for the negotiation of IKE SAs.
IPsec SA Parameters on page 29 The **ipsec_sa** section contains parameters for the negotiation of IPsec SAs.
IPsec SA Parameters Reference on page 30 The **ipsec_sa** section contains parameters for the negotiation of IPsec SAs.
Authentication Parameters Reference on page 32 The **authentication** section, which appears in the ike_sa section, contains authentication parameters for SA negotiation.
Proposal Parameters Reference on page 39 The **proposal** section, which appears in the **ike_sa** and **ipsec_sa** sections, contains proposal parameters for SA negotiation.
ROHC Configuration Parameters on page 44 The **rohc** section of the IKE configuration file, which appears in the **ipsec_sa** section, contains parameters for configuring ROHC.

2.1. General Configuration Parameters Reference

The **general** section of the IKE configuration file contains parameters that configure the IKE task.

The entries that can appear in this section are shown below and described in the table below.

The section identifier consists of the word **general**:

```
general {
    listen ipaddress [, ipadress, ...ipaddress];
    retransmits number;
    exchange_max_time number;
    spi_min number;
    spi_max number;
    default_ike_sa ike_saf
    default_ipsec_sa ipsec_sa
}
```

Table 1. General Configuration Parameters

Entry Name	Description	Default
listen	IP address of the interface at which the IKE task listens for incoming ISAKMP messages. Multiple interface addresses may be specified, separated by commas. You can use either an IP address (for example, 192.168.1.2 or 2000::79), or interface names such as eth0, eth1, eth2, etc. You can also use any, 0.0.0.0 (IPv4) or :: (IPv6) in the listen section to listen on all addresses and interfaces. The IKE daemon uses the PKTINFO from the received packets to extract the local and peer address the packet was sent with. This enables the IKE daemon to match the received packet against the IKE configuration. In order to use this feature, the IKE peer must know the address of the IKE daemon configured with any. Also, the IKE daemon configured with any should have an IKE configuration section which has the local_address parameter configured to the address on which it expects incoming packets. In the IPV6 case, where the address is not known beforehand, the configuration used by the configuration of the IKE daemon has to be updated dynamically using the configuration API, or the configuration file should include as many entries as addresses that are expected to be allocated. For more information, see Example: Using Listen on Any Address on page 72.	

Entry Name	Description	Default
	If you put an interface name, for example, eth0, in the **listen** section, IKE extracts the IPv4 address from the interface and uses that for the **listen** section. Only automatic IPV4 address retrieval is supported if you specify interfaces in the **listen** section. For example, if you specify eth0, then only IPv4 addresses are retrieved from the interfaces.	
retransmits	Maximum number of attempts to retransmit a message.	5
exchange_max_time	Maximum setup time for an exchange, in seconds.	30
spi_min	Minimum SPI value that IKE can assign to an inbound IPsec SA (the SPI of the outbound IPsec SA is generated by the IKE peer). Valid range is [1, *spi_max*].	1
spi_max	Maximum SPI value that IKE can assign to an inbound IPsec SA (the SPI of the outbound IPsec SA is generated by the IKE peer). Valid range is [*spi_min* , 4294967295].	4294967295
default_ike_sa	Section containing default values for **ike_sa** section entries. The values in this section are used if they are not specified in actual **ike_sa** sections. See IKE SA Parameters Reference on page 26.	
default_ipsec_sa	Section containing default values for **ipsec_sa** section entries. The values in this section are used if they are not specified in actual **ipsec_sa** sections. See IKE SA Parameters Reference on page 26.	

Parent topic: IKE Configuration File on page 24

2.2. IKE SA Parameters Reference

The **ike_sa** section contains parameters for the negotiation of IKE SAs.

The entries that can appear in this section are shown symbolically below and described in the table below.

The section identifier consists of the word address followed by the IP address of a remote ISAKMP/IKE task from which a connection is to be accepted. The responder may specify a network address; for example, 0.0.0.0/0 to accept connections from all IPv4 addresses or :: to accept connections from all IPv6 addresses.

Table 1. IKE SA Parameters

Entry name	Description	Default
ike_id	String that identifies this IKE SA. This ID will be used to map IPsec SAs to this IKE SA.	
exchange_version	Version of the IKE protocol to be used: 1 means v1 2 means v2	1,2

Entry name	Description	Default
	1,2 means the initiator tries to connect with v1 and the responder accepts both v1 and v2 2,1 means the initiator tries to connect with v2 and the responder accepts both v1 and v2	
exchange_mode	For IKEv1 only. Negotiation method to be used. main means main mode aggressive means aggressive mode main, aggressive means the initiator tries to connect using main mode and the responder accepts both main and aggressive modes	main, aggressive
passive	In passive mode, this host acts as the responder and does not initiate negotiation. Otherwise, this host acts as initiator; negotiation starts when the IKE task starts. Use the value off for the initiator and on for the responder. You must configure IKE as the initiator (passive parameter off) to be able to automatically rekey IPsec and IKE SAs between the peers. Currently, IKEv1 and IKEv2 can rekey IPsec SAs from either the IKE original initiator or original responder. IKEv1 does not currently support rekeying of IKE SAs. For IKEv2, rekeying of IKE SA can be initiated from either original initiator or original responder. Reauthentication in IKEv1 only reestablishes the IKE SAs. Rekeying can be initiated from any side when the lifetime expires. For IKEv1, use the *ipike.rekey_passive* sysvar to initiate an IPsec rekey. Wind River IKE does not support automatic re-negotiation of IKE SAs if you set the passive parameter to On. When the IKE peer is configured as initiator (passive parameter Off), the IKE SAs are not re-negotiated automatically for IKEv1. Only if the IPsec SAs are missing or need to be re-negotiated will the IKE SA be re-negotiated between the peers. There is no special support for reauthentication in IKE; reauthentication is achieved by performing a new IKE negotiation, that is, a re-negotiation. In IKEv2 this results in a new IKE SA with new IPsec SAs. In IKEv1 only the IKE SA is replaced.	on
initial_contact	This host may have rebooted and peers may have SAs that are no longer valid. Use the value on to send an initial contact message to a peer so that it will delete old SAs. Use the value off to disable this feature.	on
reauth	IKE SA rekeying, reauthentication option. If set to yes, the stack negotiates a new IKE SA. If set to no, the stack uses the **CREATE_CHILD_SA** message to renew the existing IKE SA.	yes.
liveness	This is a Dead Peer Detection (DPD) liveness check. This is a measurement of how long a time with an absence of incoming traffic the ike_sa accepts before sending a DPD message.	0

Entry name	Description	Default
local_address	IP address of the local ISAKMP/IKE task from which a connection is to be accepted.	First address in the ike_sa section: `ike_sa` *local_address remote_address* `{`
remote_address	IP address or range specifier of a remote IKE task from which a connection is to be accepted. The following types of addresses are valid as the **remote_address** of the ike_sa configuration: **a specific address, for example, 192.168.0.1** **an address range, for example, 192.168.0.1-192.168.0.10** **any address, for example, 0.0.0.0/0, or ::** **a subnet address, for example, 192.168.0.0/24** There is no specific precedence between the types addresses. It is the order of occurrence in the configuration that matters; that is, an incoming IKE connection is matched against the first ike_sa configuration with a **remote_address** which matches the source address of the packet.For more information and examples, see Configuring the ike_sa remote_address Selector on page 54.	Second address in the ike_sa section: `ike_sa` *local_address remote_address* `{`

local_identity	Local system identifier to be sent with ID payload during negotiation. Allowed formats of this entry are:
	`local_identifier` *type stringvalue* `;`
	`local_identifier` *type* `0x`*hexvalue* `;`
	Supported values of type are: address, address6, keyid, fqdn, user_fqdn, and asn1_dn.
		Default type is address
		Default value is the value of the local_address entry

remote_identity	Remote system identifier. Allowed formats of this entry and supported values of type are the same as for local_identifier.	Do not check value sent by peer.
lifetime	SA lifetime in time units or kilobytes, coded in the formats shown below. lifetime *value* days lifetime *value* hours lifetime *value* minutes lifetime *value* seconds lifetime *value* terabytes lifetime *value* gigabytes lifetime *value* megabytes lifetime *value* kilobytes	12 hours
authentication	Section containing authentication parameters, as described under Authentication Parameters Reference on page 32.	

Entry name	Description	Default
proposals	Section containing proposal parameters, as described under Proposal Parameters Reference on page 39.	

Parent topic: IKE Configuration File on page 24

2.3. IPsec SA Parameters

The **ipsec_sa** section contains parameters for the negotiation of IPsec SAs.

The entries that can appear in this section are shown symbolically below and described in IPsec SA Parameters Reference on page 30.

```
ipsec_sa flow {
    ike_id string;
    encapsulation_mode string;
    dynamic_address interface,type
    lifetime lifetime;
    use_tfc string;
    flowtype option;
    use_ecn option;
    granularity option;
    proposal proposal;
}
```

The section identifier consists of the word ipsec_sa followed by a flow identifier in the following format:

`address` *source_ipaddress* [*source_port*] `address` *destination_ipaddress* [*destination_port*] [*protocol*]

IP addresses may be either IPv4 or IPv6. A single address or a range of addresses are both supported for the source and destination addresses.

Allowed values for port are 1 to 65535; if port is not specified, all ports are accepted. The port can also be a range of port values when using IKEv2 (port ranges are not supported with IKEv1). Note that ports must include a protocol specification of either tcp or udp.

Allowed values for protocol are: icmp, icmpv6, udp, tcp, or any; if a protocol is not specified, all protocols are accepted. As noted above, when specifying a port or range of ports, the protocol must be either tcp or udp.

The following example shows the use of flow address / port ranges:

```
ipsec_sa address 2000::125-2000::128 50-60 address 2000::129-2000::135 50-60 tcp {
        ipsec_id ipsecsa68;
        ike_id ikesa68;
        use_tfc off;
        encapsulation_mode transport;
        proposal {
                dh_group 2;
                use_esn off;
                encryption_algorithm des;
                integrity_algorithm md5;
        }
}
```


WNDRVR

In this example:

- The source address range is from 2000::125 to 2000::128.
- The destination address range is from 2000::129 to 2000::135.
- The source port is a range from 50 to 60.
- The destination port range is also from 50 to 60.
- The port protocol is tcp.

This configuration negotiates an IPsec SA between the source address range 2000::125-2000::128 (with a source port range of 50 to 60) and destination address range 2000::129-2000:: 135 (with a destination port range of 50 to 60).

Note: Be aware that an IPsec SA configured in an IKE configuration file could overwrite an IPsec SA configured by an IPsec shell command or API call.

Parent topic: IKE Configuration File on page 24

2.4. IPsec SA Parameters Reference

The **ipsec_sa** section contains parameters for the negotiation of IPsec SAs.

Table 1. IPsec SA Parameters

Entry name	Description	Default
ike_id	String that identifies the IKE SA used to negotiate this IPsec SA. Use the ike_id string of an existing IKE SA.	
encapsulation_mode	Encapsulation mode for Phase 2 negotiation. For IP encapsulation, values are transport and tunnel . When NAT detection is enabled and Wind River IKE detects a NAT device, it automatically switches from transport or tunnel to udp_transport or udp_tunnel, respectively. (These values are still available for backward compatibility.)	tunnel
dynamic_address	Instructs the initiator to retrieve the IPsec flow address from the responder. Value has the following format: interface,type. interface is an interface identifier such as eth0. type is either ip4 for an IPv4 address or ip6 for IPv6 .	Default type is none (static address)
lifetime	SA lifetime in time units or kilobytes, coded in the formats shown below. lifetime value days lifetime value hours lifetime value minutes lifetime value seconds lifetime value terabytes lifetime value gigabytes lifetime value megabytes lifetime value kilobytes	8 hours

Entry name	Description	Default
	For IPsec SA negotiation, you can code two lifetime parameters to specify size and time; for example: lifetime 1000 kilobytes lifetime 100 hours	
use_tfc	Specifies the use of TFC. Values are On or Off. The default is Off.	Off
flowtype	This parameter determines whether IP traffic for a tunnel should be dropped if the tunnel is down or forwarded without IPsec protection. Valid options for this setting are: acquire, require, or none. If you choose acquire or require, this parameter adds an egress flow, with the specified require or acquire option, to protect the outgoing packets to be sent if there are no corresponding flowtypes. When this is set to flowtype require and the tunnel is down, the packet is dropped and the IKE daemon is notified. When this is set to flowtype acquire and the tunnel is down, the acquire flow sends the packet in clear text and notifies the IKE daemon. If none is selected, no additional flow is created. The packet is forwarded without using the tunnel (unencrypted).	
use_ecn	This parameter manages the Explicit Congestion Notification (ECN) setting for the ipsec_sa. It can be set to On or Off. ECN is a mechanism for indicating the occurrence of congestion between tunnel endpoints. ECN is turned on default for all IKE (both IKEv1 and IKEv2) negotiated IPsec SAs. For manually keyed IPsec SAs added by the keyadm command the default setting is off, but it can be turned on by specifying the-ecn option. For more information about keyadm, see the *VxWorks 7 IPsec Programmer's Guide*. If ECN is enabled on the IPsec SA, it will copy CE (Congestion Experienced) from outer header to inner header at decapsulation if and only if CE is set on outer header and the inner header is ECN-capable ECT(0) or ECT(1) (ECT is ECN-Capable Transport). See RFC 4301 for more information and details. If ECN is disabled on the IPsec SA the decapsulator mechanism will leave the inner ECN fields untouched.	On
granularity	Two types of granularity are implemented: flow Uses the flow parameters given in the configuration as SA selectors. host Uses the host addresses from the trigger packet as SA selectors	
proposals	Section containing proposal parameters, as described under Proposal Parameters Reference on page 39.	

Parent topic: IKE Configuration File on page 24

2.5. Authentication Parameters Reference

The **authentication** section, which appears in the ike_sa section, contains authentication parameters for SA negotiation.

The entries that can appear in this section are shown symbolically below and described in the table below.

```
authentication {
    method method;
    psk_id string;
    shared_key string;
    certificate file_name;
    certificate_uri http_uri_path;
    send_certificate boolean;
    onrequest boolean;
    request_certificate boolean;
    remote_filename file_name;
    ocsp_response string;
    key_id id;
    private_key file_name;
    private_key_password string;
    trusted_ca_store directory_name;
    crl_store uri_path;
    intermediate_ca_store directory_name;
    identity identity;
}
radius {
    server_address ipaddress;
    local_address ipaddress;
    shared_secret string;
}
```

Either key_id *id* or private_key *fileName* should be given in the **authentication** section, and either psk_id *id* or shared_key *key* should be given. If both are given in the **authentication** section, the id parameter will take precedence.

Table 1. Authentication Parameters

Entry name	Description	Default
method	Authentication method. Allowed values are pre_shared_key , rsa_signature, dss_signature. Multiple methods may be specified, separated by commas. The host uses the first value specified.	
psk_id	This identifies the buffer ID used to fetch the pre-shared secret from the Secure Key database.	
shared_key	Used by method pre_shared_key. This is a string containing the shared secret.	
certificate	Used by methods rsa_signature and dss_signature . Path of certificate file for the public key of the local host. The public key in the certificate must correspond to the private key pointed to by private_key.	

WNDRVR

Entry name	Description	Default

certificate_uri	Specifies the base location of the certificate that together with the SHA1 hash forms the complete URI pointing out the certificate. For example:
	`certificate_uri http://windriver.com/certificates/;`
	or
	`certificate_uri http://172.16.0.11/user/certs/`
	or
	`certificate_uri http://2001:1::5/;`	

send_certificate	Specifies whether to send the certificate. By default, the certificate is sent.	"yes"
onrequest	Specifies whether to send the certificate when it has been requested. That is, send it when a certificate payload is received.	
request_certificate	Specifies whether to send a certificate request. By default, a certificate request is is sent.	"yes"
remote_filename	The filename of the remote peer's certificate. If a filename is not specified, the certificate is loaded from the configuration.	
ocsp_response	Wind River IKE can be configured to request and supply OCSP content using the configuration directive **ocsp_response** in the **authentication** section of the IKE SA. The **ocsp_response** directive points to the PEM encoded OCSP response which validates the certificate configured by the certificate directive.	
key_id	The **key_id** is used as key ID when fetching an RSA/DSA key from the Secure Key database.	
private_key	Used by methods rsa_signature and dss_signature. Path of PEM-encoded private key file. The key must correspond to the public key in the certificate pointed to by certificate.	
private_key_password	Used by methods rsa_signature and dss_signature. Password used to encrypt the private key. Only needed if you have not changed the default private key password callback to implement a custom way of obtaining the private key password. See **src/ipike_h.h** for a definition of the callback.	
trusted_ca_store	Used by methods rsa_signature and dss_signature. Directory containing certificates for all public keys that are directly trusted by the system.	

Entry name	Description	Default
intermediate_ca_store	Used by methods rsa_signature and dss_signature. Directory containing certificates for certificate authorities that are not directly trusted by the system, but might be needed when building a certificate path from a client certificate to a trusted public key. The directory can be empty if all systems share the same trusted public key.	
crl_store	A directory or URI specifying the location of a CRL.	
identity	Used for IKEv2/EAP. This contains the EAP identity.	

RADIUS Authentication Parameters Reference on page 34 The radius section, which appears in the **authentication** section of the ike_sa, contains RADIUS authentication parameters for SA negotiation.
CRL Support on page 34 Wind River IKE verifies the validity of a given certificate against a Certificate revocation list, issued by the appropriate Certificate Authority.
OCSP Configuration on page 36 Wind River IKE can be configured to request and supply OCSP content using the configuration directive **ocsp_response** in the **authentication** section of the IKE SA.
Authentication Using EAP on page 37 The Extensible Authentication Protocol (EAP) is available in IKEv2.

Parent topic: IKE Configuration File on page 24

RADIUS Authentication Parameters Reference

The radius section, which appears in the **authentication** section of the ike_sa, contains RADIUS authentication parameters for SA negotiation.

The entries that can appear in this section are described in the table below.

Entry name	Description	Default
server_address	The address of the RADIUS server.	
local_address	The local address to use for the RADIUS connection.	
shared_secret	Shared secret to use in negotiations.	

Parent topic: Authentication Parameters Reference on page 32

CRL Support

Wind River IKE verifies the validity of a given certificate against a Certificate revocation list, issued by the appropriate Certificate Authority.

The location of the CRL store is specified in the **authentication** section of the IKE SA.

For example, in the configuration snippet given below, the **crl_store** entry points to the directory **certificates/crl**, which is expected to contain the CRLs in a PEM encoded format. Wind River IKE will make use of the CRLs to verify the validity of the certificates used for authentication. The crl_store entry is only valid when certificate based authentication is used.

```
        authentication {
                        method dss_signature;
                        certificate certificates/dsacert.pem;
                        private_key keys/dsapriv.key;
                        private_key_password 1111;
                        trusted_ca_store certificates/trusted;
                        crl_store certificates/crl;
                }
```

Example IKE Configuration File Using CRL Authentication

```
# ike1.cfg
general {
        listen 192.168.1.243;
        exchange_max_time 30;
        spi_min 10000;
        spi_max 20000;

}
ike_sa address 192.168.1.244 {
        ike_id ikesa68;
        exchange_version 2;
        passive off;
        authentication {
                method dss_signature;
                certificate dsacert.pem;
                private_key dsapriv.key;
                private_key_password 1111;
        trusted_ca_store trusted;
                crl_store crl;
        }
        proposal {
                dh_group 2;
                hash_algorithm md5;
                encryption_algorithm aes128;
                integrity_algorithm md5;
        }
}
ipsec_sa address 192.168.1.243 address 192.168.1.244 {
        ipsec_id ipsecsa68;
        ike_id ikesa68;
        use_tfc off;
        lifetime 3 minutes;
        encapsulation_mode transport;
        proposal {
                dh_group 2;
                use_esn off;
                encryption_algorithm des;
                integrity_algorithm md5;
        }
}
```

Parent topic: Authentication Parameters Reference on page 32

OCSP Configuration

Wind River IKE can be configured to request and supply OCSP content using the configuration directive **ocsp_response** in the **authentication** section of the IKE SA.

The **ocsp_response** directive points to the PEM encoded OCSP response which validates the certificate configured by the certificate directive (in the example given below, dsacert.pem). The file dsacert.nok is signed by the CA that has issued the certificate dsacert.pem. This OCSP response is sent by Wind River IKE to the peer when it receives a **CERTREQ** (with **OCSP_CONTENT**) for the dsacert.pem certificate. The peer then verifies the OCSP response as well as the dsacert.pem before accepting the certificates as valid for authentication purposes.

```
                authentication {
                        method dss_signature;
                        certificate certificates/dsacert.pem;
                        private_key keys/dsapriv.key;
                        private_key_password 1111;
                        trusted_ca_store certificates/trusted;
                        ocsp_response ocsp/dsacert.nok;
                }
```

Example IKE Configuration File Using OCSP Authentication

The sample IKE configuration file below shows the use of OCSP for authentication.

```
# ike1.cfg
general {
        listen 192.168.1.243;
        exchange_max_time 30;
        spi_min 10000;
        spi_max 20000;

}
ike_sa address 192.168.1.244 {
        ike_id ikesa68;
        exchange_version 2;
        passive off;
        authentication {
                method dss_signature;
                certificate dsacert.pem;
                private_key dsapriv.key;
                private_key_password 1111;
                trusted_ca_store trusted;
                ocsp_response ocsp/dsacert.nok;
        }
        proposal {
                dh_group 2;
                hash_algorithm md5;
                encryption_algorithm aes128;
                integrity_algorithm md5;
        }
}
ipsec_sa address 192.168.1.243 address 192.168.1.244 {
        ipsec_id ipsecsa68;
        ike_id ikesa68;
        use_tfc off;
        lifetime 3 minutes;
```

```
        encapsulation_mode transport;
        proposal {
                dh_group 2;
                use_esn off;
                encryption_algorithm des;
                integrity_algorithm md5;
        }
}
```

Parent topic: Authentication Parameters Reference on page 32

Authentication Using EAP

The Extensible Authentication Protocol (EAP) is available in IKEv2.

EAP-IKEv2 provides mutual authentication and session key establishment between an EAP peer and an EAP server. It supports authentication based on the following types of credentials:

- asymmetric key pairs, where the public key is in a certificate
- passwords
- symmetric keys

To specify the use of EAP-IKEv2 authentication in Wind River IKE, you must define the IPIKE_USE_EAP parameter in the IKE initialization code. For more information about EAP, see the *Wind River EAP Programmer's Guide*. Examples of Wind River IKE configuration files that use EAP authentication follow.

Example EAP Initiator Configuration

To adapt the following example initiator IKE configuration file:

1. Update initiator to the IP address of the initiator system.
2. Update the responder address to the IP address of the responder system.
3. Change the authentication method to the desired authentication method, **pre_shared_key**, **rsa_signature**, or **dss_signature**.
4. Update the certificate to the file name of the certificate used for the authentication method **rsa_signature** or **dss_signature**.
5. Update the **private_key** to the file name of the private key used for the authentication method **rsa_signature** or **dss_signature**.

```
general {
        listen 192.168.1.141;
        exchange_max_time 30;
        spi_min 4096;
        spi_max 8191;
}
ike_sa address 192.168.1.142 {
        ike_id default1;
        local_address 192.168.1.141;
        exchange_version 2,1;
        passive off;
        liveness 0;
        initial_contact off;
        lifetime 8 hours;
```


WNDRVR

```
        authentication {
                method eap,rsa_signature;
                certificate ike_rsa_init.cer;
                private_key ike_rsa_init.key;
                private_key_password 1111;
                trusted_ca_store ./trusted;
                intermediate_ca_store ./intermediate;
                identity kalle;
        }

        proposal {
                dh_group 2,1;
                hash_algorithm sha1;
                encryption_algorithm aes128;
                integrity_algorithm sha1;
        }
}
ipsec_sa address 192.168.1.141 address 192.168.1.142 {
        ike_id default1;
        encapsulation_mode transport;
        lifetime 3600 seconds;
        proposal {
                dh_group 2,1;
                encryption_algorithm aes128;
                integrity_algorithm sha1;
        }
}
```

Example EAP Responder Configuration

To adapt the following example responder IKE configuration file:

1. Change the initiator to the IP address of the initiator system.
2. Change the responder to the IP address of the responder system. Update the authentication method with the desired authentication method, **pre_shared_key**, **rsa_signature**, or **dss_signature**.
3. Change certificate to the file name of the certificate used for the authentication method **rsa_signature** or **dss_signature**.
4. Change the private_key to the file name of the private key used for the authentication method **rsa_signature** or **dss_signature**.

```
general {
        listen 192.168.1.142;
        exchange_max_time 30;
        spi_min 4096;
        spi_max 8191;
}
ike_sa address 192.168.1.141 {
        ike_id default1;
        local_address 192.168.1.142;
        exchange_version 2,1;
        passive on;
        liveness 0;
        initial_contact off;
        lifetime 8 hours;
        authentication {
```

```
                method rsa_signature,eap;
                certificate ike_rsa_resp.cer;
                private_key ike_rsa_resp.key;
                private_key_password 1111;
                trusted_ca_store ./trusted;
                intermediate_ca_store ./intermediate;
                identity kalle;
        }
        radius {
                shared_secret kallekula123;
                server_address 192.168.1.1;
        }
        proposal {
                dh_group 2,1;
                hash_algorithm sha1;
                encryption_algorithm aes128;
                integrity_algorithm sha1;
        }
}
ipsec_sa address 192.168.1.142 address 192.168.1.141 {
        ike_id default1;
        encapsulation_mode transport;
        lifetime 3600 seconds;
        proposal {
                dh_group 2,1;
                encryption_algorithm aes128;
                integrity_algorithm sha1;
        }
}
```

Parent topic: Authentication Parameters Reference on page 32

2.6. Proposal Parameters Reference

The **proposal** section, which appears in the **ike_sa** and **ipsec_sa** sections, contains proposal parameters for SA negotiation.

The entries that can appear in this section are shown symbolically below and described in the table *Proposal Parameters*.

```
proposal {
    dh_group string[,string,...,string];
    hash_algorithm string[,string,...,string];
    encryption_algorithm string[,string,...,string];
    integrity_algorithm string[,string,...,string];
    use_esn string
    v1_lifetime lifetime;
}
```

For the IPsec SA **proposal** section, for both IKEv1 and IKEv2, the hash algorithm is not required in the configuration file or it can be set to none. It is not used during IPsec SA setup.

For IKEv1 and IKEv2, the following parameters must be included in the IKE SA proposal section and cannot be set to none.

Required IKEv1 proposal section Parameters

- hash_algorithm

- encryption_algorithm
- dh_group

Required IKEv2 proposal section Parameters

- hash_algorithm
- integrity_algorithm (authentication algorithm for generating the digest for IKE SA exchanges)
- encryption_algorithm
- dh_group

For IPsec SAs, the encryption algorithm can be null.

To negotiate an AH SA, the encryption algorithm directive must be left out.

The values any, null, and none can be used as follows:

- any can be used with encryption algorithm
- any can be used with authentication method
- any can be used with hash algorithm
- any can be used with integrity/authentication algorithm
- null can be used with encryption algorithm to specify null encryption
- none can be used with dh_group

Table 1. Proposal Parameters

Entry name	Description	Default
dh_group	Group used for Diffie-Hellman negotiations. Allowed values are as follows: **modp768 modp1024** **modp1536** **modp2048** **modp3072** **modp4096** **modp6144** **modp8192** Alternatively, you can also specify the corresponding dh_group numbers: 1, 2, 5, 14, 15, 16, 17, and 18, respectively.	
hash_algorithm	Hash algorithm to be used to generate key material for ike_sa negotiation. The following hash algorithms are supported in the ike_sa proposal: **md5** **sha1** **aes128** **any**	

Entry name	Description	Default
	sha256 **sha384** **sha512** For aes128 , you must set the ipike.nonce_size to aes128 keysize. Multiple algorithms may be specified, separated by commas. In IKEv1, the initiator uses only the first algorithm; the responder can use all algorithms in the list. In IKEv2, both initiator and responder can use all algorithms in the list.	
encryption_algorithm	Encryption algorithm to be used to protect data packets. The following encryption algorithms are supported in the **ike_sa** proposal: **des** **3des** **blowfish** **cast** **aes128cbc** **aes192cbc** **aes256cbc** **any** The following encryption algorithms are supported in the **ipsec_sa** proposal: **des** **3des** **blowfish** **cast** **aes128cbc** **aes192cbc** **aes256cbc** **aes128ctr** **aes192ctr** **aes256ctr** **any** **null** Multiple algorithms may be specified, separated by commas. In IKEv1, the initiator uses only the first algorithm; the responder can use all algorithms in the list. In IKEv2, both initiator and responder can use all algorithms in the list. Note that ALG support depends on the underlying Wind River Cryptography Libraries support, and not all ALGs may be supported for all IKE versions.	

Entry name	Description	Default
integrity_algorithm	Integrity algorithm to be used to protect data packets. The following integrity algorithms are supported in the **ike_sa** proposal: **md5** **sha1** **any** **sha256** **sha384** **sha512** The following integrity algorithms are supported in the **ipsec_sa** proposal: **md5** **sha1** **aes** **any** **sha256** **sha384** **sha512** Multiple algorithms may be specified, separated by commas. IKEv1 does not make use of an integrity algorithm for the IKE traffic; IKEv2 does. A configured integrity algorithm in the **ike_sa** proposal will be ignored if IKEv1 is used. In IKEv2, both initiator and responder can use all algorithms in the list.	
use_esn	Specifies the use of 64-bit ESN. Valid values are: on or off . When set to off, there is no replay protection. ESN must be set to On to enable anti-replay. This parameter applies only to IPsec SAs and should only be used in the **ipsec_sa proposal** section.	on
v1_lifetime	SA lifetime in time units or kilobytes, using the formats shown below. lifetime *number* hours lifetime *number* minutes lifetime *number* seconds lifetime *number* Kb In IKEv1, you can specify a different lifetime for each proposal. This parameter has no effect in IKEv2.	8 hours

ESP and AH Support in a Single Proposal on page 43 Wind River IKEv1 supports accepting both ESP and AH negotiation in the same proposal for a given IPsec flow configuration.

Parent topic: IKE Configuration File on page 24

ESP and AH Support in a Single Proposal

Wind River IKEv1 supports accepting both ESP and AH negotiation in the same proposal for a given IPsec flow configuration.

This is required for interoperability with Windows Vista and Windows Server equivalents which allow such policy specifications. ESP and AH in a single proposal is done by specifying the IPsec policy in the following manner:

```
ipsec_sa address 192.168.133.125 address 192.168.133.129 {
        ipsec_id ipsecsa128;
        ike_id ikesa68;
        lifetime 600 seconds;
        encapsulation_mode transport;
        use_tfc off;
        proposal {
                number 1;
                dh_group modp1024;
                encryption_algorithm 3des;
                integrity_algorithm md5;
                use_esn off;
        }
        proposal {
                number 1;
                dh_group modp1024;
                integrity_algorithm md5;
                use_esn off;
        }
}
```

Versions of Wind River IKEv1 previous to release 6.8 did not support specification of two proposals with the same number (note that the proposal numbers must be the same, otherwise they are considered to be different proposals).

Also, the order of proposals is important in order to interoperate with Vista. Vista requires the ESP followed by AH to be applied to the packets and therefore in the Wind River IKE policy specification the ESP proposal should come before the AH specification, as shown in the example above. The DH group specifications are required if PFS must be enabled between the peers.

For Vista IPsec configuration, refer to the Microsoft supplied documentation related to the Vista MMC (Microsoft Management Console). Using the Vista MMC, you can configure both the certificates and IPsec policy, and you can monitor the status of the connections.

Note: Note that because Vista does not support the ESN and TFC features, they must be explicitly turned off in the Wind River IKE configuration.

The following is an example IKE configuration for Vista interoperability.

```
# Windows Vista
general {
        listen 192.168.133.125;
        exchange_max_time 30;
        spi_min 10000;
        spi_max 20000;

}
ike_sa address 192.168.133.129 {
        ike_id ikesa68;
        exchange_version 1;
        passive off;
```

```
        lifetime 3 minutes;
        use_tfc off;
        authentication {
                method rsa_signature;
                certificate ike_new_rsa_init.cer;
                private_key ike_new_rsa_init.key;
                private_key_password 1111;
                trusted_ca_store trusted;
        }
        proposal {
                dh_group modp1024;
                hash_algorithm md5;
                encryption_algorithm 3des;
                integrity_algorithm md5;
        }
}
ipsec_sa address 192.168.133.125 address 192.168.133.129 {
        ipsec_id ipsecsa128;
        ike_id ikesa68;
        lifetime 600 seconds;
        encapsulation_mode transport;
        use_tfc off;
        proposal {
        number 1;
                dh_group modp1024;
                encryption_algorithm 3des;
                integrity_algorithm md5;
                use_esn off;
        }
        proposal {
        number 1;
                dh_group modp1024;
                integrity_algorithm md5;
                use_esn off;
        }
}
```

Parent topic: Proposal Parameters Reference on page 39

2.7. ROHC Configuration Parameters

The **rohc** section of the IKE configuration file, which appears in the **ipsec_sa** section, contains parameters for configuring ROHC.

profiles

This parameter enables the appropriate ROHC profiles on the IPsec SA. The profiles are configured as comma separated lists. For example, the following line enables the UDP IP and IP compression scheme in ROHC:

```
profiles 2,4;
```

For supported profiles, see the *VxWorks 7 Network Stack Programmer's Guide*.

integrity_algorithm

An integrity algorithm can be negotiated to ensure that a decompressed header is identical to the original header. This is done by calculating an integrity check value (ICV) over the uncompressed packet.

Supported algorithms are md5, sha1, aes, sha256, sha384, and sha512. This parameter is optional.

WNDRVR

icv_len

The number of bytes of the ICV that are used and transmitted with the compressed packet. icv_len is optional. It must be greater than zero to enable the integrity value (ICV).

Note: There are two additional parameters that can be signaled with IKE, max_cid and mrru. However, these parameters are not currently used.

For an example configuration, see ROHCoIPsec-Processed Packet Example on page 88.

Parent topic: IKE Configuration File on page 24

3. Programming the Initialization Code

The Wind River IKE initialization code initializes the IKE task during the VxWorks kernel startup.

The initialization code consists of the following files:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ike/h/ipike_config.h**

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ike/cfg/ipike_config.c**

This section describes sysvar settings, parameter defines, and function calls that you can program into the initialization code to configure Wind River IKE at startup.

Note: If you modify the initialization code, you must recompile Wind River IKE as instructed in the *VxWorks 7 Configuration and Build Guide*.

Setting Sysvars Reference on page 45
Setting Parameter Defines Reference on page 48 Various IKE parameters are not accessible through Workbench. To set these parameter defines, edit **ipike_config.h**.
Calling Configuration Functions on page 50 Wind River IKE includes a configuration API that sets many of the same parameters as the configuration file.

3.1. Setting Sysvars Reference

The following table lists the IKE sysvars, which are set in the *ike_conf_sysvar* array in **ipike_config.c**, and their equivalent Workbench parameters, which are in the VxWorks image project (VIP).

Table 1. Sysvars and Equivalent Workbench Configuration Parameters

Sysvar and Equivalent Workbench Parameter	Description
ipike.create_default_conf IPIKE_CREATE_DEFAULT_CONFIG_FILE	Create a default configuration file. (The default configuration allows the host to act as a responder and instructs the authentication to use a preshared key.)
ipike.dynamic_address.mode IPIKE_DYNAMIC_ADDRESS_MODE	For IKEv1 only. The dynamic address mode tells the initiator how to set addresses in the IPsec source flow. When the initiator requests a dynamic address, it sets the scope of the IPsec source flow according to this value. This parameter affects all SAs.

Sysvar and Equivalent Workbench Parameter	Description
	In IKEv1, the initiator has to set the flow addresses because the responder is not allowed to change the flow. For example, a VPN client may be allowed to connect directly to a particular server, or it may be required to tunnel all traffic through the VPN gateway. The dynamic address mode tells the client how to set addresses in the flow. 0 means set IPsec source address to "any" address. 1 means set IPsec source address to the received address. 2 means set IPsec source address to the received network.
ipike.address_pool.dns4 IPIKE_DYNAMIC_ADDRESS_POOL_DNS4	When the initiator requests a dynamic address, the responder uses this IPv4 address to contact a domain name server to obtain information to send to the initiator.
ipike.address_pool.network4 IPIKE_DYNAMIC_ADDRESS_POOL_NETWORK4	When the initiator requests a dynamic address, the responder uses this IPv4 address with a subnet mask to generate information to send to the initiator. For example, if the address is 10.1.111.1/24, the responder first sends 10.1.111.1, then 10.1.111.2, and so on up to 10.1.111.254. The subnet mask is 255.255.255.0.
ipike.pfkeyv2.flush_shutdown IPIKE_FLUSH_SHUTDOWN	Set to 1 to flush keys at shutdown.
ipike.address_pool.anyflow IPIKE_DYNAMIC_ADDRESS_ANYFLOW	Set this flag to 1 to make the responder traffic selector match any address in the case of dynamic addresses with IKEv2 (responder).
ipike.address_pool.homeaddr IPIKE_DYNAMIC_ADDRESS_HOMEADDR	Set this to 1 to add the initiator's dynamic address as the home address (initiator).
ipike.pfkeyv2.flush_startup IPIKE_FLUSH_STARTUP	Set to 1 to flush keys at startup.
ipike.pfkeyv2.flush_reconfigure IPIKE_FLUSH_RECONFIGURE	Set to 1 to flush keys on reconfiguration.
ipike.port_nat IPIKE_NAT_PORT	UDP port on which the IKE task s for ISAKMP NAT-Traversal (NAT-T) packets. Set to port 4500 to enable NAT-T detection. Default is 0 (feature disabled).
ipike.port IPIKE_PORT	UDP port on which the IKE task s for ISAKMP packets.
ipike.resend_delay IPIKE_RESEND_DELAY	IKE message resend delay, in seconds. This is the time that the IKE task is to wait before attempting the first resend of a packet. IKE waits twice this interval between the first and second attempts, three times this interval between the second and third resend, and so on.
ipike.isakmp_max_exchanges IPIKE_MAX_EXCHANGES	Maximum number of active ISAKMP exchanges. When the maximum is reached, packets for new exchanges will be dropped. Set to zero to allow unlimited exchanges.
ipike.filename	Name and path of the IKE configuration file.

Sysvar and Equivalent Workbench Parameter	Description
IPIKE_CONFIG_FILE_PATH IPIKE_CONFIG_FILE_NAME	
ipike.nonce_size IPIKE_NONCE_SIZE	Size in bytes of the nonce (random number) that the IKE task generates during Phase 1 exchange. Must be in the range 8 to 256 bytes.
ipike.isakmp_sock_buf_size IPIKE_SOCK_BUF_SIZE	ISAKMP socket buffer size, in bytes. This size determines the largest ISAKMP message that the IKE task can send and receive.
ipike.keepalive_nat IPIKE_NAT_KEEPALIVE	Interval between NAT-Traversal keepalive packets, in seconds.
ipike.pfkeyv2.soft_to_hard_time IPIKE_SOFT_TO_HARD_TIME	Number of seconds that the soft timeout (and the renewal negotiation) should be before the hard timeout occurs.
ipike.use_hash_and_url IPIKE_USE_HASH_AND_URL	Enables support of the "Hash and URL" feature. The default setting is disabled, since there is a potential security risk to accept and retrieve a URL from an unauthenticated source.
ipike.http_request_timeout IPIKE_HTTP_REQUEST_TIMEOUT	Maximum number of seconds to wait for an HTTP response. The default is 5 seconds.
ipike.http_response_buf_size IPIKE_HTTP_RESPONSE_BUF_SIZE	Maximum size of an HTTP response that can be received. The default is 4096 bytes.
ipike.rekey_fuzziness IPIKE_REKEY_FUZZINESS	This parameter is used for avoiding situations where both IKE peers initiate a rekey exchange simultaneously. A value of 20 delays the rekey timer with a random amount of time between 0 and 20 seconds. The rekey fuzziness applies on both IKE SA and IPsec SA rekeys. A value of 0 disables the fuzziness. Note that both the IKE SA and IPsec SAs are re-established when the IKE SA expires in IKEv1.
ipike.mobike No equivalent Workbench parameter.	Set to 1 to enable the mobility and multihoming feature of Wind River IKE. Default is 0 (feature disabled).
ipike.rekey_passive No equivalent Workbench parameter.	For IKEv1 only. When an IPsec or IKE SA expires, the original initiator usually initiates a rekeying negotiation. This sysvar is intended for use when interfacing with an IKE implementation that cannot initiate rekeying. Set to 1 to cause the original responder to initiate rekeying. Default is 0 (feature disabled).
ipike.address_pool.network6 No equivalent Workbench parameter.	When the initiator requests a dynamic address, the responder uses this IPv6 address to generate information to send to the initiator. An example of how to set this sysvar in the IPCOM shell is: sysvar set -o -c ipike.address_pool.network6 2001::1/64 In this example, Wind River IKE assigns addresses starting from 2001::1, 2001::2, and so on to 2001::64.

Sysvar and Equivalent Workbench Parameter	Description
ipike.liveness_timer_resolution IPIKE_LIVENESS_TIMER_RESOLUTION	This controls how often all (inbound) IPsec SAs should be queried for activity, that is, if they have received any packets. This sysvar only controls how often the IPsec SAs are polled in order to have a fresh "last used" value. The default value is 10 seconds. A lower value makes the DPD trigger more accurate, but is also more costly when it comes to CPU usage.
pike.max_ike_sa_num IPIKE_MAX_IKE_SA_NUM	Maximum number of IKE SAs. This sysvar specifies the maximum number of concurrent IKE SAs that can be established. Set to 0 to allow infinite number of IKE SAs. Default is 0 .
ipike.max_ipsec_sa_num IPIKE_MAX_IPSEC_SA_NUM	Maximum number of IPsec SAs. This sysvar specifies the maximum number of concurrent IPsec SAs that can be established. Set to 0 to allow infinite number of IPsec SAs. Default is 0 .
ipike.max_message_size IPIKE_MAX_MESSAGE_SIZE	Maximum size of an IKE message. This sysvar specifies the maximum size of an IKE message. This restriction will be applied on both outgoing and incoming IKE traffic. Set to 0 to for unlimited size (limited by the maximum UDP packet size). Default is 0 .
ipike.sa_simultaneous_estab_period IPIKE_SA_SIMULTANEOUS_ESTABLISHMENT_PERIOD	IKE simultanous establishement period. This sysvar specifies the time in seconds for ike simultanous establishement period. If two or more IKE SAs from the same remote peer are established within this period of seconds, they will be treated as a simultaneous (collision) exchange and the IKE SA containing the smallest nonce will be removed. Set to 1 to for 1 second period. Default is 1.

Note:

Rekeying and reauthentication of SAs is done by having overlapping SAs for a short time frame. This needs to be taken into account when using the SA limits. Also note that the IPsec SAs are typically installed in pairs, one SA for outgoing traffic and one SA for incoming traffic. For example, if *ipike.max_ipsec_sa_num* =2, then it would reject IPsec SA rekeying attempts because new IPsec SAs cannot be installed.

Parent topic: Programming the Initialization Code on page 45

3.2. Setting Parameter Defines Reference

Various IKE parameters are not accessible through Workbench. To set these parameter defines, edit **ipike_config.h**.

IPIKE_MAX_LISTEN_ADDRS

Sets the maximum number of addresses. The default value is 8.

IPIKE_SYSLOG_PRIORITY

Sets the minimum priority level that a log entry must have to be included in the build. Entries with a lower level will not be included in the build.

WNDRVR

IPIKE_USE_VERSION2

This parameter enables support for IKEv2. It cannot be undefined to remove support for IKEv2. It should always be defined.

IPIKE_USE_EAP

Define this parameter to enable support for EAP. This parameter is automatically defined if IPEAP is included in the build. For information about EAP, see the *Wind River EAP Programmer's Guide*.

IPIKE_USE_RADIUS

Define this parameter to enable support for RADIUS in EAP. This parameter is automatically defined if both IPEAP and IPRADIUS are included in the build.

IPIKE_USE_ROHC

Define this parameter to enable ROHC parameter negotiation using IKE version 2. This parameter is automatically defined if both IPIKE_USE_VERSION2 and IPROHC are included in the build.

IPIKE_USE_AGGRESSIVE_MODE

Define this parameter to enable aggressive mode negotiation for IKE version1

IPIKE_REAUTH_MARGIN_SEC

The number of seconds before the IKE SA expires that reauthentication should take place. The default is 20 seconds.

IPIKE_REKEY_MARGIN_SEC

The number of seconds before the IKE SA expires that rekey should take place. By default, it is double IPIKE_REAUTH_MARGIN_SEC.

IPIKE_REKEY_RETRY_TIMEOUT_MS

The number of milliseconds to wait before retrying a rekey when the IKE SA is busy (that is, an exchange is in progress). This timeout is also used for the same purpose when performing a reauhtentication. The default value is 1000 milliseconds or 1 second.

IPIKE_USE_CRYPTO_SELF_TEST

Define this parameter to enable cryptography self test on startup. This parameter is defined automatically if you define IP_DEBUG and IPIKE_USE_VERSION2.

IPIKE_INFORMATIONAL_RETRY_TIMEOUT_MS

The number of milliseconds to wait before retrying an information exchange when the IKE SA is busy (that is, an exchange in progress). The default value is 1000 milliseconds or 1 second.

IPIKE_INFORMATIONAL_RETRY_QUEUE_SIZE

Specifies how many information exchange requests to save in a queue when the IKE SA is busy (that is, an exchange in progress). The default size is 10.

IPIKE_USE_AES_MAC

Define this parameter to use AES encryption. This parameter is automatically defined if OPENSSL_NO_AES is not defined and IPCRYPTO is defined.

IPIKE_SYSLOG_PRIORITY

The default syslog priority. This is set to _WRS_CONFIG_IPIKE_SYSLOG_PRIORITY if _WRS_CONFIG_IPIKE_SYSLOG_PRIORITY is defined. Otherwise, this is set to IPCOM_SYSLOG_PRIO_DEFAULT.

WNDRVR

IPIKE_USE_PKTINFO

Define this parameter to use PKTINFO processing to listen on :: or 0.0.0.0. This can be only used if IPNET is defined. It is automatically defined if IPNET is defined.

IPIKE_USE_PERF_EVAL

Define this parameter to enable the performance evaluation code for IKE version 1 and 2.

IPIKE_USE_OCSP

Define this parameter to use OCSP request/responses to verify certificate validity status. This parameter is defined automatically if OPENSSL_NO_OCSP is not defined and IPCRYPTO is defined.

IPIKE_DPD_WINDOW_SIZE

Specifies the size of the DPD window which contains acceptable sequence numbers. This is for IKEv1 only. The default value is 10.

IPIKE_IKEV2_MAX_EXCH_IN_NEG

Specifies the maximum number of exchanges that are allowed to complete an IKE version 2 negotiation. An IKE version 2 negotiation is normally completed within 2 exchanges (one INIT and one AUTH), but if EAP is used as authentication method, more exchanges are required. One exchange consists of one request-response message pair. This is for IKE version 2 only. The default value is 15.

IPIKE_IKE_SA_SIMULTANEOUS_ESTABLISHMENT_PERIOD

If two or more IKE SAs from the same remote peer are established within this period of seconds, they will be treated as a simultaneous (collision) exchange and the IKE SA containing the smallest nonce is removed. The default value is 1 second.

IPIKE_DISPATCHER_RECEIVE_RETRIES

Specifies the number of times the IKE dispatcher will tolerate receive errors before it reconfigures itself by re-opening the listening UDP sockets. The default value is 3.

IPIKE_CONFIG_ROHC_INTEG_NUM_MAX

Specifies the maximum number of integrity algorithms that can be configured and proposed when ROHC is enabled. The default value is 7.

IPIKE_CONFIG_ROHC_PROFILE_NUM_MAX

Specifies the maximum number of ROHC profiles that can be configured.The default value is 8.

IPIKE_ROHC_IFNAME_SIZE

Specifies the ROHC interface name size. The default value is 16.

Parent topic: Programming the Initialization Code on page 45

3.3. Calling Configuration Functions

Wind River IKE includes a configuration API that sets many of the same parameters as the configuration file.

You can program **ipike_config.c** to call these functions. For reference documentation on the configuration API, see the online Wind River IPSec and IKE API reference.

Parent topic: Programming the Initialization Code on page 45

4. Configuring Cryptographic Suites for VPNs

Wind River IPsec implements the VPN-A and VPN-B suites as described in RFC 4308.

These cryptographic suites are for use in IPsec virtual private networks. The following sections describe how to configure VPN suites for IKE SAs. For information on configuring VPN suites for IPsec SAs, see the *VxWorks 7 IPsec Programmer's Guide*.

Configuring the VPN-A Suite on page 51 The VPN-A suite is intended to match the currently most commonly used VPN security settings in IKEv1.
Configuring the VPN-B Suite on page 52 The VPN-B suite comprises settings that many expect to be the commonly used VPN security settings within a few years.

4.1. Configuring the VPN-A Suite

The VPN-A suite is intended to match the currently most commonly used VPN security settings in IKEv1.

RFC 4308 defines the following algorithms for the VPN-A suite:

- Encryption - TripleDES in CBC mode [RFC 2451]
- Pseudo-random function - HMAC-SHA1 [RFC 2104]
- Integrity - HMAC-SHA1-96 [RFC 2404]
- Diffie-Hellman group - 1024-bit Modular Exponential (MODP) [RFC 2409]

IKEv2 Configuration

The following example IKE configuration file shows the use of the VPN-A suite in the proposal section. Note that the exchange_version can be set to 1 for IKEv1 for VPN-A suite configuration.

```
general {
        listen 14.15.16.17;
}
ike_sa address 22.23.24.25 {
                ike_id default
                exchange_version 2;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group modp1024;
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
                lifetime 86400 seconds;
}
ipsec_sa address 0.0.0.0/0 address 172.23.9.0/24 {
                ike_id default;
                use_tfc off;
                lifetime 28800 seconds;
                proposal {
                                dh_group modp1024;
                                encryption_algorithm 3des;
```

```
                                integrity_algorithm sha1;
                                use_esn off;
                }
}
```

Parent topic: Configuring Cryptographic Suites for VPNs on page 51

4.2. Configuring the VPN-B Suite

The VPN-B suite comprises settings that many expect to be the commonly used VPN security settings within a few years.

RFC 4308 defines the following algorithms for the VPN-B suite:

- Encryption - AES with 128-bit keys in CBC mode [AES-CBC]
- Pseudo-random function - AES-XCBC-PRF-128 [AES-XCBC-PRF-128]
- Integrity - AES-XCBC-MAC-96 [AES-XCBC-MAC]
- Diffie-Hellman group - 2048-bit MODP [RFC3526]

IKEv2 Configuration

The following example IKE configuration file shows the use of the VPN-B suite in the proposal section.

```
general {
                listen 114.15.16.17;
}
ike_sa address 22.23.24.25 {
                ike_id default;
                exchange_version 2;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group modp2048;
                                hash_algorithm aes128;
                                encryption_algorithm aes;
                                integrity_algorithm aes;
                }
                lifetime 86400 seconds;
}
ipsec_sa address 0.0.0.0/0 address 172.23.9.0/24 {
                ike_id default;
                use_tfc off;
                lifetime 28800 seconds;
                proposal {
                                dh_group modp2048;
                                encryption_algorithm aes;
                                integrity_algorithm aes;
                                use_esn off;
                }
}
```

Parent topic: Configuring Cryptographic Suites for VPNs on page 51

5. Dead Peer Detection (DPD)

Functional Overview

Wind River IKE supports Dead Peer Detection (DPD) in both IKEv1 and IKEv2.

The IKE daemon initiates DPD if the time since the last received packets is greater than or equal to the liveness setting (configured per ike_sa) at every poll of the IPsec SA. Note that the IKE daemon will not initiate DPD at any time except at the time of polling the IPsec SA; when IKE polls the IPsec SA, if the time since the last received packets (both IKE and IPsec packets) is greater than or equal to the liveness setting, it will initiate DPD.

Both received IPsec traffic and received IKE traffic count as a proof of liveness. In other words, DPD will never be initiated as long as IPsec traffic or IKE packets are received from the peer.

If no DPD response is received within configured **exchange_max_time** or the maximum number of retransmits the IKE SA and all IPsec SAs set up through that **ike_sa** are deleted.

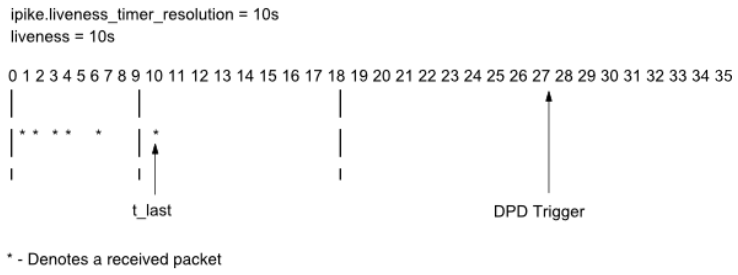
A new negotiation is triggered either with outgoing traffic or when the house keeping timer kicks in.

Configuring DPD

You configure DPD by setting the liveness parameter in the ike_sa configuration file and by setting the sysvar *ipike.liveness_timer_resolution* .

The sysvar *ipike.liveness_timer_resolution* controls how often all (inbound) IPsec SAs should be queried for activity, that is, if they have received any packets. The liveness parameter in the ike_sa configuration file is a measurement of how long a time with an absence of incoming traffic the ike_sa accepts before sending a DPD message.

Note that the *ipike.liveness_timer_resolution* sysvar only controls how often the IPsec SAs are polled in order to have a fresh "last used" value. The default value is 10 seconds. A lower value makes the DPD trigger more accurately but is also more costly when it comes to CPU usage. The *ipike.liveness_timer_resolution* should always be configured to be less than or equal to the configured liveness value in order to get the desired result. The following diagram is an example of polling every 10 seconds with liveness configured to 10 seconds:



In the example diagram above, the last received packet was at t_last=11s, and you might think that DPD will be triggered at t=21s, but since the IPsec SAs are polled every 10 seconds, it will not be detected before t=30s. If the resolution of the timer instead had been configured to 1 it would have been possible to trigger DPD at t=21s.

6. Configuring Rekeying

Wind River IKE provides configuration settings and parameters for use in rekeying.

These parameters enable you to configure rekeying of SAs in a manner that minimizes packet loss and maximizes compatibility.

The rekeying of SAs is based on the effects of the settings of the following parameters in the IKE configuration file and the following IKE component parameters:

- ike_sa lifetime setting(s)
- ike_sa reauth (for IKE SA rekeying in IKEv2)
- ipsec_sa lifetime setting(s)
- ipsec dh_group setting (PFS)
- IPIKE_SOFT_TO_HARD_TIME

The parameters IPIKE_MAX_IKE_SA_NUM and IPIKE_MAX_IPSEC_SA_NUM affect rekeying, depending on their configuration. That is, they must be set to allow for the overlapping of SAs. The parameters can be set at build time or at run time using the corresponding sysvars.

7. Configuring the ike_sa remote_address Selector

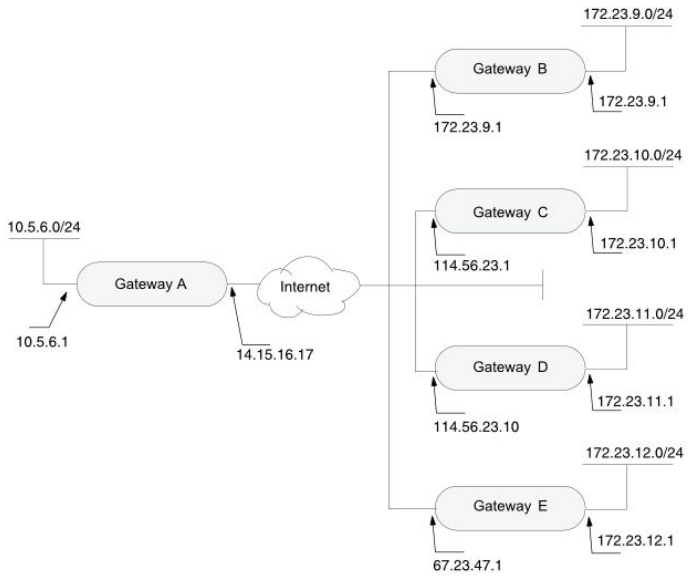
The **ike_sa** section of the Wind River IKE configuration file includes a remote_address parameter for identifying a remote peer.

The remote_address can be configured to accept a specific address, an address range, any address, or a subnet.

The following example use case configurations illustrate the use of each type of setting for the remote_address parameter.

Note that the IKE peer selectors for IPV4 and IPV6 addresses behave similarly.

Based on the figure below, the following example use case IKE configurations accept incoming traffic on the WAN interface 14.15.16.17 on Gateway A.



Example 1: Specific Address

The following IKE configuration file example shows the use of a SPECIFIC address for the remote_address parameter.

```
general {
         listen 14.15.16.17;
}
ike_sa address 172.23.9.1 {
                ike_id default;
                exchange_version 2;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group modp1024;
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
                lifetime 86400 seconds;
}
ipsec_sa address 14.15.16.17 address 172.23.9.1-172.23.9.9 {
                ike_id default;
                use_tfc off;
                lifetime 28800 seconds;
                proposal {
                                dh_group modp1024;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                                use_esn off;
                }
}
```

Example 2: Range Address

The following IKE configuration file example shows the use of a RANGE of addresses for the remote_address parameter.

```
general {
                listen 14.15.16.17;
}
ike_sa address 114.56.23.0-114.56.23.10 {
                ike_id default;
                exchange_version 2;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group modp1024;
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
```

```
                }
                lifetime 86400 seconds;
}
ipsec_sa address 14.15.16.17 address 114.56.23.0-114.56.23.10 {
                ike_id default;
                use_tfc off;
                lifetime 28800 seconds;
                proposal {
                                dh_group modp1024;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                                use_esn off;
                }
}
```

Example 3: Any Address

The following IKE configuration file example shows the use of an ANY address specifier for the remote_address parameter.

```
general {
                listen 14.15.16.17;
}
ike_sa address 0.0.0.0/0 {
                ike_id default;
                exchange_version 2;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group modp1024;
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
                lifetime 86400 seconds;
}
ipsec_sa address 14.15.16.17 address 0.0.0.0/0 {
                ike_id default;
                use_tfc off;
                lifetime 28800 seconds;
                proposal {
                                dh_group modp1024;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                                use_esn off;
                }
}
```

Example 4: Subnet Address

The following IKE configuration file example shows the use of a SUBNET address specifier for the remote_address parameter.

```
general {
                listen 14.15.16.17;
}
ike_sa address 172.23.0.0/24 {
                ike_id default;
                exchange_version 2;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group modp1024;
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
                lifetime 86400 seconds;
}
ipsec_sa address 14.15.16.17 address 172.23.0.0/24 {
                ike_id default;
                use_tfc off;
                lifetime 28800 seconds;
                proposal {
                                dh_group modp1024;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                                use_esn off;
                }
}
```

8. Configuring IPsec Granularity

The first element of the flowtype feature allows IKE to start a negotiation with a peer using the destination address of the packet that matches the IPsec trigger flow.

This means that the IKE remote peer address does not need to be known at configuration time. For example:

```
ike_sa address 192.168.0.0/24 {
...
}

ipsec_sa address 192.168.1.0/24 address 192.168.0.0/24 {
flowtype require;
...
}
```

The configuration above would install a “trigger flow” with 192.168.1.0/24 -> 192.168.0.0/24 addresses. If a packet is matched against the addresses in the “trigger flow”, that is, 192.168.1.1 -> 192.168.0.2, it would initiate an IKE negotiation to 192.168.0.2.

The second part of the flowtype require feature is the granularity setting. This setting controls the traffic selector granularity of the installed IPsec SA. Looking at the example configuration above, IKE would install an IPsec SA (and use flows) with the network addresses in the configuration, that is, 192.168.1.0/24 <-> 192.168.0.0/24. This would mean that all traffic destined to 192.168.0.0/24 would use the same SA.

By setting the granularity setting to host it is possible to limit the traffic selectors of the IPsec SA to addresses of the packet that once triggered the IKE negotiation, that is, 192.168.1.1/32 <-> 192.168.0.1/32. A packet destined for 192.168.0.2 would not be matched by such an SA and would instead trigger a new IKE negotiation towards 192.168.0.2.

Two types of gran*flow*ularity are implemented:

Uses the flow parameters given in the configuration as SA selectors

host

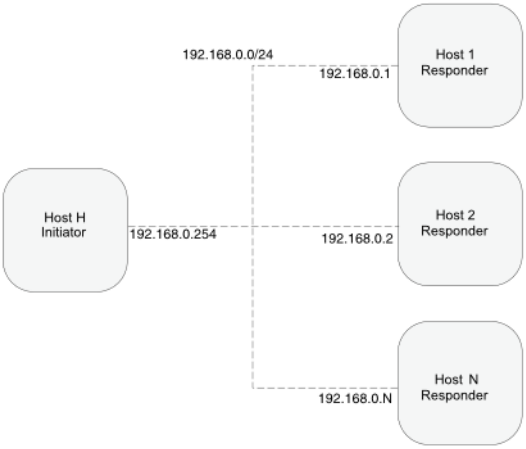
Uses the host addresses from the trigger packet as SA selectors

The setting is defined in the **ipsec_sa** section of the IKE configuration file:

```
ipsec_sa address 192.168.1.0/24 address 192.168.0.0/24 {
flowtype require;
granularity host;
...
}
```

9. IPsec Granularity Use Case Scenario

A use case scenario helps to illustrate the purpose and use of the granularity parameter.



If Host H must communicate with any of the Hosts 1 to N it must do so securely using IPsec. One way of achieving this would be to configure a separate IPsec SA (with corresponding IKE SA sections) in the IKE configuration file for each Host. For example:

Tunnel for Host 1

```
ipsec_sa address 192.168.0.254/32 address 192.168.0.1/32 {
        ike_id ike_host1;
        lifetime 900 seconds;
        encapsulation_mode transport;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                use_esn off;
```

```
        }
}
```

Tunnel for Host 2

```
ipsec_sa address 192.168.0.254/32 address 192.168.0.2/32 {
        ike_id ike_host2;
        lifetime 900 seconds;
        encapsulation_mode transport;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                use_esn off;
        }
}
```

This results in a large static configuration file and configuration. A more efficient way of accomplishing this would be to use the flowtype setting in the IPsec SA configuration section. The following configuration would accomplish the same thing as the previous two example configurations.

Tunnel for Host X

```
ipsec_sa address 192.168.0.254/32 address 192.168.0.0/24 {
        ike_id ike_hostX;
        lifetime 900 seconds;
        encapsulation_mode transport;
        flowtype require;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                use_esn off;
        }
}
```

This configuration installs a “trigger flow” that results in an IPsec tunnel that is negotiated if traffic matches the flow defined by the address settings 192.168.0.254/32 -> 192.168.0.0/24. For example, if Host H sends traffic to Host 1 (192.168.0.1) it would match the trigger flow and initiate an IKE negotiation to Host 1.

While this seems acceptable, there is one problem with the configuration above. The IPsec SAs which are installed will contain the traffic selectors 192.168.0.254/32 and 192.168.0.0/24. This means that traffic destined to Host 2 will also match the selectors of the IPsec SAs and tunneled to Host 1. This is not desirable. The solution for this problem is to use the granularity setting, which controls the granularity of the traffic selectors of the IPsec SAs. Two types of SA granularity are currently supported, host and flow.

The flow granularity setting will use (copy) the addresses found in the trigger flow. The host granularity setting will copy the addresses found in the addresses of the triggering packet. So by adding the setting granularity host to the configuration above we get the desirable behavior.

Tunnel for Host X

```
ipsec_sa address 192.168.0.254/32 address 192.168.0.0/24 {
        ike_id ike_hostX;
        lifetime 900 seconds;
        encapsulation_mode transport;
```

```
        flowtype require;
        granularity host;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                use_esn off;
        }
}
```

Now if Host H sends traffic to Host 1 it will construct IPsec SAs with traffic selectors 192.168.0.254/32-192.168.0.1/32. If Host H then wants to communicate with Host 2, it will not match the same IPsec SA, but it will match the trigger flow, which results in a new IPsec SA pair with traffic selectors 192.168.0.254/32-192.168.0.2/32.

10. Configuring Event Callback Logging

The use of IKE/IPsec event logging is controlled at build time with the VSB component **COMPONENT_IPIKE_USE_EVENT_LOG**, which is disabled by default.

To enable event callback logging, include **COMPONENT_IPIKE_USE_EVENT_LOG** in a VSB and then base your VxWorks image project on that VSB.

When enabled, the **COMPONENT_IPIKE_USE_EVENT_LOG** component activates event logging in both IPsec and IKE. When this component is disabled it is completely removed from the binaries. Note that individual log events can be enabled and disabled in:

installDir **/pkgs_v2/net/ipnet/ipsecike**_x.x.x.x_ **/ike/h/ipike_config.h**

The following events can be triggered:

- **IPIKE_EVENT_SOFT_LIFE_SIZE_EXPIRED**
- **IPIKE_EVENT_SOFT_LIFE_TIME_EXPIRED**
- **IPIKE_EVENT_HARD_LIFE_SIZE_EXPIRED**
- **IPIKE_EVENT_HARD_LIFE_TIME_EXPIRED**
- **IPIKE_EVENT_IKE_PACKET_DISCARD**
- **IPIKE_EVENT_IKE_PHASE_I_FAIL**
- **IPIKE_EVENT_IKE_PHASE_I_SUCCESS**
- **IPIKE_EVENT_IKE_PHASE_II_FAIL**
- **IPIKE_EVENT_IKE_PHASE_II_SUCCESS**
- **IPIKE_EVENT_IPSEC_PACKET_DISCARD**
- **IPIKE_EVENT_SEQUENCE_NUM_OVERFLOW**
- **IPIKE_EVENT_SPD_PACKET_DISCARD**

Each event is connected with an event reason, either **IPIKE_REASON_UNDEFINED**, or one of the following specific descriptions:

- **IPIKE_REASON_UNDEFINED**
- **IPIKE_REASON_AUTHENTICATION_ERROR**
- **IPIKE_REASON_DECRYPTION_ERROR**
- **IPIKE_REASON_ENCRYPTION_ERROR**
- **IPIKE_REASON_EXPLICIT_DISCARD_POLICY**
- **IPIKE_REASON_INVALID_SPI_ERROR**
- **IPIKE_REASON_NEGOTIATION_TIMEOUT**

- **IPIKE_REASON_NO_POLICY**
- **IPIKE_REASON_NO_PROTECTION_SUITE**
- **IPIKE_REASON_POLICY_ERROR**
- **IPIKE_REASON_REPLAY_ERROR**
- **IPIKE_REASON_IPSEC_INTERNAL_ERROR**
- **IPIKE_REASON_ISAKMP_PACKET_ERROR**
- **IPIKE_REASON_PROPOSAL_MISMATCH**
- **IPIKE_REASON_PSK_MISMATCH**
- **IPIKE_REASON_FLOW_MISMATCH**
- **IPIKE_REASON_IKE_INTERNAL_ERROR**
- **IPIKE_REASON_IKE_SA_OVERFLOW**
- **IPIKE_REASON_IPSEC_SA_OVERFLOW**

The message passing between the IKE daemon and the library relies on TCP. This gives a connection oriented channel which can be used across process boundaries. All events triggered in IPsec are relayed via the IKE daemon using pfkey before it arrives in the library.

It is currently only possible to use the library within a VIP project, that is, it will not build inside a DKM project or an RTP project.

Event Callback Logging APIs

The public API functions available to a user are:

```
IP_PUBLIC int
ipike_event_receiver_init(Ipike_event_receiver_cb event_cb);

IP_PUBLIC void
ipike_event_receiver_destroy(void);
```

ipike_event_receiver_init( ) replaces the two previously existing functions, ipsecLoggerInit( ) and ipsecLoggerCallbackSet( ). At the library initialization, a new task is spawned to handle incoming events from the IKE daemon.

ipike_event_receiver_destroy( ) signals the receiving task to finish and clean up any resources held by the library.

Event Callback Logging Example Code

```
#include <ipike_event_receiver.h>

int event_cb(Ipike_event *event)
{
        printf("Got event\n", );
        printf(" type: %s\n", ipike_event_type_str(event->type));
        printf(" reason: %s\n", ipike_event_reason_str(event->reason));
        return 0;
}

void example_start(void)
        {
        /* Initialize event receiver library */
        ipike_event_receiver_init(event_cb);
        }
```

```
void example_stop(void)
        {
        /* Stop and destroy event receiver */
        ipike_event_receiver_destroy();
        }
```

5. EXAMPLE IMPLEMENTATIONS

1. Introduction

Learn about typical implementations of Wind River IKE, the setup of a LAN connection and a gateway-to-gateway VPN.

You can use these examples to start development according to your own requirements. This chapter also includes example code fragments showing common use cases for the IKE configuration API.

In order to utilize some of the configuration examples in this section, you must include the following components in your VxWorks image project:

- **INCLUDE_KEYADM_CMD**
- **INCLUDE_IPSECCTRL_CMD**

2. General Procedures
2.1. Booting the Target

About This Task

Procedure

1. Place the VxWorks image and the startup script on an FTP server that is accessible from the target.
2. From a host such as a PC, open a serial console session to COM1 of the target.
3. Switch on the target. The target downloads the VxWorks image and startup script. The image loads into memory and the startup script runs.

   The target downloads the VxWorks image and startup script. The image loads into memory and the startup script runs.

   The following appear on the console:

   1. listing of boot parameters
   2. attaching to network interface card
   3. downloading VxWorks image from FTP server
   4. initializing Wind River IKE
   VxWorks banner
   5. running Wind River IKE startup script

2.2. Changing Target Boot Parameters

You can change boot parameters such as the address of the FTP server at tun-time.

About This Task

Perform this step if you want to change any of the target's boot parameters, such as the IP address of the FTP server.

Procedure

1. Start the boot process as described above.

   If the target is already running, reboot it by pressing ctrl-x on the console.

   The auto-boot message appears after a few seconds.

2. When the prompt "Press any key to stop auto-boot..." appears, press any key to stop the boot process.

   You have eight seconds to press a key, after which the boot process resumes.

3. At the "[VxWorks Boot]:" prompt, type p to list the boot parameters.

   Following are brief descriptions of some of the parameters:

   file name

   VxWorks image to be downloaded to target

   inet on ethernet

   public IP address of target

   host inet

   IP address of FTP server

   user

   user name for accessing FTP server

   ftp password

   password for accessing FTP server

   startup script

   configuration script to be run after boot

4. Type c to start changing parameters.
5. Press Enter multiple times to list through the parameters. Pressing Enter leaves the parameter value unchanged.

   To change a parameter value, type the new value and press Enter. When all changes are done and the "[VxWorks Boot]:" prompt appears, type p to check the new parameter values. The target saves the new values so that they will be used at the next boot.

6. Type @ to resume the boot process.

2.3. Rebooting the Target

It may be necessary to reboot the target between tests, so that the target can be in a known state.

About This Task

Procedure

To reboot, press ctrl-x on the console.

WNDRVR

The auto-boot message appears after a few seconds.

2.4. Logging On to the Target

You can connect to the target using Telnet or transfer files using FTP.

For more details about Telnet and FTP, see the *VxWorks 7 Network Stack Programmer's Guide*.

Telnet connects to the VxWorks shell using port 23. The default username and password for the VxWorks shell are set at build time. Telnet connects to the IPCOM shell using port 2323. By default, the username and password are not set.

By default, the FTP server on the target has the username **anonymous** with no password. This is a read-only account.

Note: If the target runs IPsec but the host from which you connect to the target does not, you must disable output packet filtering on the target.

3. IKE Configuration API Examples

3.1. Example: Setting Up IPsec/IKE SAs

Create an IPsec tunnel between a source and destination.

The tunnel source and destination are *Tunnelsrc* and *Tunnelend*, respectively. The corresponding IKE SA and then the IPSEC SAs and flows are also created.

```
Ipike_error create_tunnel( char * dst_addr, char * src_addr, char * IfGW, char * pw,  char * IfLoc
EP )
{
        Ipike_error                ret;
        Ipike_config_api_general   general;
        Ipike_config_api_ike_sa    ike;
        Ipike_config_api_ipsec_sa  ipsec;
        Ipike_config_api_proposal  proposal;
        int                        proposal_id;
        void *                     ike_config_ctx = NULL;
        char                       buf1[64];
        char                       buf2[64];
        char                       ike_id[IPIKE_CONFIG_MAX_ID_LEN];
        char                       ike_id_back[IPIKE_CONFIG_MAX_ID_LEN];
        char                       ipsec_id[IPIKE_CONFIG_MAX_ID_LEN];
        struct       Ip_in_addr    sin_addr_dst;
        struct       Ip_in_addr    sin_addr_src;
        int                        mask_dst;
        int                        mask_src;
        char*                      pos;
        int                        position;

        Ip_bool                    ike_sa_exists=FALSE;

   /* IKE context has to be created only once for all the tunnels */
        if (first_tunnel)
        {
                first_tunnel = FALSE;
                ike_config_ctx = ipike_config_api_new(FALSE);
```

```
                ipcom_memset(&general, 0, sizeof(general));

                /* Setup General section */
                general.listen = IfLocEP;
                general.retransmits = 5;
                general.exchange_max_time = 2;
                general.spi_max = 0xffffffff;
                general.spi_min = 1;

                ret = ipike_config_api_set_general(ike_config_ctx,&general);

        }
        else
        {
                /* Load existing IKE config */
                ike_config_ctx = ipike_config_api_new(TRUE);
        }

        if (strcmp(dst_addr,src_addr)==0)
        {
                return IPIKE_FAILED;
        }

        /* Check if ike sa already exists using IKE id */
        ipcom_sprintf(ike_id, "%s%s", IfLocEP, IfGW);

        position = IPIKE_CONFIG_INVALID_VALUE;
        while (ipike_config_api_get_next_ike_sa(ike_config_ctx, ike_id_back, &ike, &position )== I
PIKE_SUCCESS)
        {
                if (ipcom_strcmp(ike_id, ike_id_back) == 0)
                {
                        ike_sa_exists = TRUE;
                }
                ipike_config_api_release_ike_sa(&ike);
        }

        if (!ike_sa_exists)
        {
                ipcom_memset(&ike, 0, sizeof(ike));

                ike.version = "2";

                /* Initiator vs Responder */
                if (is_initiator)
                {
                        ike.passive = FALSE;/*FALSE for initiator*/
                }
                else
                {
                        ike.passive = TRUE;/*TRUE for responder*/
                }

                ike.liveness = 0;

                /*Create remote address string*/
                ipcom_sprintf(buf1, "address %s", IfGW);
```

```
                ike.remote_address = buf1;
                ike.local_address = IfLocEP; /*local GW*/
                /*Create local identifier string*/
                ipcom_sprintf(buf2, "address %s", IfLocEP);
                ike.local_id = buf2;

                /* Setup authentication method */
                ike.authentication.method = "pre_shared_key";
                ike.authentication.shared_key = pw;

                /*Setup up IKE SA config*/

                ret = ipike_config_api_set_ike_sa(ike_config_ctx,ike_id,&ike);

                ipcom_memset(&proposal, 0, sizeof(proposal));
                /* Setup IKE proposal */
                proposal.dh_group = "2";
                proposal.hash_alg = "sha1,md5";
                proposal.encryption_alg = "aes256";
                proposal.integrity_alg = "sha1";
                proposal_id = IPIKE_CONFIG_INVALID_VALUE;

                ret = ipike_config_api_set_ike_sa_proposal(ike_config_ctx,ike_id, &proposal_id, &p
roposal);
                if (ret !=0)
                {
                        ipcom_printf("ipike_config_api_set_ike_sa_proposal failed %d\n", ret);
                }

        }

        ipcom_memset(&ipsec, 0, sizeof(ipsec));

        /*Create unique IPSEC id */
        if (ipcom_inet_pton(IP_AF_INET, dst_addr, &sin_addr_dst) == 0)
        {
                printf("Bad destination address\n");
                return IPIKE_FAILED;
        }

        if (NULL != (pos = ipcom_strchr(dst_addr, (int)'/')))
                mask_dst = ipcom_atoi(pos+1);
        else
                mask_dst=32;

        if (ipcom_inet_pton(IP_AF_INET, src_addr, &sin_addr_src) == 0)
        {
                printf("Bad source address\n");
                return IPIKE_FAILED;
        }

        if (NULL != (pos = ipcom_strchr(src_addr, (int)'/')))
                mask_src = ipcom_atoi(pos+1);
        else
                mask_src=32;

        ipcom_sprintf(ipsec_id, "%x%x_%x%x", sin_addr_dst.s_addr,mask_dst, sin_addr_src.s_addr, ma
```

```
sk_src );

        /* Configure IPSEC SA */
        ipsec.ike_id = ike_id;
        ipsec.encap_mode = "tunnel";
        ipsec.lifetime = "3600 seconds";
        ipsec.flow.src_address = src_addr; /*src*/
        ipsec.flow.dst_address = dst_addr; /*dst*/

        ret = ipike_config_api_set_ipsec_sa(ike_config_ctx, ipsec_id, &ipsec);
        if (ret !=0)
        {
                ipcom_printf("ipike_config_api_set_ipsec_sa failed %d\n", ret);
        }

        /* IPSEC SA proposal */
        ipcom_memset(&proposal, 0, sizeof(proposal));
        proposal.dh_group = "2";
        proposal.encryption_alg = "aes256";
        proposal.integrity_alg = "sha1";
        proposal_id = IPIKE_CONFIG_INVALID_VALUE;

        ret = ipike_config_api_set_ipsec_sa_proposal(ike_config_ctx, ipsec_id, &proposal_id, &prop
osal);
        if (ret !=0)
        {
                ipcom_printf("ipike_config_api_set_ipsec_sa_proposal failed %d\n", ret);
        }

        ret = ipike_config_api_commit(ike_config_ctx);
        if (ret !=0)
        {
                ipcom_printf("ipike_config_api_commit failed %d\n", ret);
        }
        ipcom_sleep(10); /* Delay for the IKE exchanges to occur */
        ipike_config_api_free(ike_config_ctx);

return( ret );
}
```

3.2. Example: Removing IPsec/IKE SAs

Tear down a tunnel between a source and destination.

The corresponding IKE SA and IPsec SAs and flows are removed.

```
Ipike_error delete_tunnel( char * dst_addr, char * src_addr, char * IfGW, char * IfLocEP )
{
        Ipike_error                ret;
        int                        pos;
        char                       ike_id[IPIKE_CONFIG_MAX_ID_LEN];
        char                       ike_id_back[IPIKE_CONFIG_MAX_ID_LEN];
        char                       ipsec_id[IPIKE_CONFIG_MAX_ID_LEN];
        char                       ipsec_id_back[IPIKE_CONFIG_MAX_ID_LEN];
        void *                     ike_config_ctx = NULL;
        struct      Ip_in_addr     sin_addr_dst;
```

```
        struct      Ip_in_addr     sin_addr_src;
        int                        mask_dst=32;
        int                        mask_src=32;
        char*                      ppos;

        Ipike_config_api_ipsec_sa  ipsec;
        Ipike_config_api_ike_sa    ike;
        Ip_bool                    ipsec_ike_sa_exists=FALSE;

        /*Source and destination cannot be same*/
        if (strcmp(dst_addr,src_addr)==0)
        {
                return IPIKE_FAILED;
        }
        ike_config_ctx = ipike_config_api_new(TRUE);

        /*Generate unique ipsec_id based on src_addr and dst_addr*/
        if (ipcom_inet_pton(IP_AF_INET, dst_addr, &sin_addr_dst) == 0)
        {
                printf("Bad destination address\n");
                return IPIKE_FAILED;
        }

        if (NULL != (ppos = ipcom_strchr(dst_addr, (int)'/')))
                mask_dst = ipcom_atoi(ppos+1);

        if (ipcom_inet_pton(IP_AF_INET, src_addr, &sin_addr_src) == 0)
        {
                printf("Bad source address\n");
                return IPIKE_FAILED;
        }

        if (NULL != (ppos = ipcom_strchr(src_addr, (int)'/')))
                mask_src = ipcom_atoi(ppos+1);

        ipcom_sprintf(ipsec_id, "%x%x_%x%x", sin_addr_dst.s_addr,mask_dst, sin_addr_src.s_addr, ma
sk_src );
        ipcom_sprintf(ike_id, "%s%s", IfLocEP, IfGW);

        pos = IPIKE_CONFIG_INVALID_VALUE;
        while (ipike_config_api_get_next_ipsec_sa(ike_config_ctx, ipsec_id_back, &ipsec, &pos) ==
IPIKE_SUCCESS)
        {
                /*Find out is this ipsec sa is last one for ike sa*/
                /*if yes - delete ike sa as well*/
                if ((ipsec_id_back[0] != 0)
                        && (ipcom_strcmp(ipsec.ike_id, ike_id) == 0)
                        &&(ipcom_strcmp(ipsec_id_back, ipsec_id) != 0))
                {
                        ipsec_ike_sa_exists=TRUE;
                }

        (void)ipike_config_api_set_ipsec_sa(ike_config_ctx, ipsec_id, NULL);
                ipike_config_api_release_ipsec_sa(&ipsec);
        }

        if (!ipsec_ike_sa_exists)
```

```
        {
                pos = IPIKE_CONFIG_INVALID_VALUE;
                while (ipike_config_api_get_next_ike_sa(ike_config_ctx, ike_id_back, &ike, &pos) =
= 0)
                {
                        if( ike_id_back[0] &&  !(ipcom_strcmp(ike_id_back, ike_id)))
                        {
                                ipcom_printf("Deleting IKE SA ok %s \n",ike_id);

                                ret = ipike_config_api_set_ike_sa(ike_config_ctx,ike_id,NULL);

                                if (ret !=0)
                                {
                                        ipcom_printf("ipike_config_api_set_ike_sa failed %d\n", re
t);
                                }

                                ipike_config_api_release_ike_sa(&ike);

                        }

                }


        }

        ret = ipike_config_api_commit(ike_config_ctx);
        ipike_config_api_free(ike_config_ctx);
        if (ret !=0)
        {
                ipcom_printf("ipike_config_api_commit failed %d\n", ret);
        }

        return ret;

}

/* IKE initiator */
void ike_initiator(void)
{

        is_initiator=TRUE;

        create_tunnel( "192.168.1.247/32", "192.168.1.249/32", "192.168.1.247", "topsecret", "192.
168.1.249" );

        if(!check_tunnel("192.168.1.247/32", "192.168.1.249/32",TRUE))
                goto quit;

        delete_tunnel( "192.168.1.247/32", "192.168.1.249/32", "192.168.1.247", "192.168.1.249" );

        if(check_tunnel("192.168.1.247/32", "192.168.1.249/32",FALSE))
        goto quit;
quit:
        ipcom_printf(" Quitting due to tunnel creation/deletion  failure \n");

}
```

```
/* IKE Responder  */
void ike_responder(void)
{

        is_initiator=FALSE;

        create_tunnel( "192.168.1.249/32", "192.168.1.247/32", "192.168.1.249", "topsecret", "192.
168.1.247" );


        if(!check_tunnel("192.168.1.249/32", "192.168.1.247/32", TRUE ))
                goto quit;

quit:
        ipcom_printf(" Quitting due to tunnel creation failure \n");
}
```

3.3. Example: Check for IPSEC/IKE SA Existence

Check if the IPSEC SAs exist or do not exist between a source and destination.

While this example code is included to show how the IKE configuration API can be used to check for the existence of SAs, it is recommended that this be done using the PFKEYV2 API instead.

```
Ip_bool check_tunnel( char * dst_addr, char * src_addr, Ip_bool exists )
{
        char         ipsec_id[IPIKE_CONFIG_MAX_ID_LEN];
        struct       Ip_in_addr          sin_addr_dst;
        struct       Ip_in_addr          sin_addr_src;
        int          mask_dst;
        int          mask_src;
        char*        ppos;
        Ip_bool      test;
        int          retry = 5;

        /*Generate IPSEC ID*/
        if (ipcom_inet_pton(IP_AF_INET, dst_addr, &sin_addr_dst) == 0)
        {
                printf("Bad destination address\n");
                return IPIKE_FAILED;
        }

        if (NULL != (ppos = ipcom_strchr(dst_addr, (int)'/')))
                mask_dst = ipcom_atoi(ppos+1);

        if (ipcom_inet_pton(IP_AF_INET, src_addr, &sin_addr_src) == 0)
        {
                printf("Bad source address\n");
                return IPIKE_FAILED;
        }

        if (NULL != (ppos = ipcom_strchr(src_addr, (int)'/')))
                mask_src = ipcom_atoi(ppos+1);

        ipcom_sprintf(ipsec_id, "%x%x_%x%x", sin_addr_dst.s_addr,mask_dst, sin_addr_src.s_addr, ma
```

```
sk_src );


        /*Check if IPSEC SA exists in SADB */
        if(exists == TRUE)
        {
                test = FALSE;
                while(!test && retry)
                {
                        ipcom_printf("Retrying to check SA existence %d.. \n",retry);
                        IPIKE_CODE_LOCK();
                        test=ipike_ipsec_sadb_sa_exist(ipsec_id);
                        IPIKE_CODE_UNLOCK();
                        ipcom_sleep(5);
                        retry--;
                }
        }
        else
        {
                test = TRUE;
                while(test && retry)
                {
                        ipcom_printf("Retrying to check SA removal %d.. \n",retry);
                        IPIKE_CODE_LOCK();
                        test=ipike_ipsec_sadb_sa_exist(ipsec_id);
                        IPIKE_CODE_UNLOCK();
                        ipcom_sleep(5);
                        retry--;
                }


        }

        return test;

}
```

3.4. Example: Using Listen on Any Address

Use the setting *ANY* in the general section of the IKE configuration file to listen to any address.

The example below illustrates the use of the setting *ANY* in the general section of the IKE configuration file to listen to any address. Note the use of an address in the **ike_sa address** section. In order to use this feature the IKE peer must know the address of the IKE daemon configured with any. The IKE daemon configured with any should also have an IKE configuration section which has the local_address parameter configured to the address on which it expects incoming packets. In the IPV6 case, where the address is not known beforehand, the configuration used for the configuration of the IKE daemon has to be updated dynamically using the configuration API, or the configuration file should include as many entries as addresses that are expected to be allocated.

```
general {
        listen ::;
        exchange_max_time 30;
        spi_min 10000;
        spi_max 20000;
}
ike_sa address 2000::7 {
        ike_id ikesa68;
```

```
        exchange_version 2;
        local_address 2000::6;
        passive on;
        lifetime 15 minutes;
        use_tfc off;
        initial_contact on;
        authentication {
                method pre_shared_key;
                shared_key this_should_be_secret;
        }
        proposal {
                dh_group 14;
                hash_algorithm md5;
                encryption_algorithm aes128;
                integrity_algorithm md5;
                use_esn off;
        }
}
```

4. Examples: Configurations for IP Multipoint VPN Access

Learn from configuration files (three different initiators and one responder) that support IP multipoint VPN access.

The most common scenario where it is useful to distinguish clients (initiators) based on their ID is when the client has obtained its IP address dynamically. If the responder does not know which address the client can connect from, it will be forced to accept incoming IKE connections from any IP address (0.0.0.0/0).

Distinguishing IKE connections by ID is also useful when you want to have different access policies for different clients, for example, client1@wrs.com could only access network 192.168.1.0/24, while client2@wrs.com could only access network 192.168.2.0/24.

Note that the initiator's ID is not sent before the second message exchange in IKEv2 or IKEv1 using Main mode. This requires that all proposal parameters sent in the first message exchange must be the same for all ike_sa configurations that accept IKE negotiations from any (0.0.0.0/0) IP address.

Initiator 1

```
general {
        listen 10.7.1.2;
        exchange_max_time 10;
        retransmits 3;
        spi_min 1000;
        spi_max 9999;
}
ike_sa address 10.7.1.1 {
        ike_id ike_sa_1;
        passive off;
        local_address 10.7.1.2;
        local_identifier user_fqdn init1@windriver.com;
        exchange_version 2;
        reauth no;
        lifetime 90 seconds;
        authentication {
                method pre_shared_key;
```

```
                shared_key shared_secret_1;
        }
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                hash_algorithm sha1;
                dh_group modp1024;
                use_esn off;
        }
}
ipsec_sa address 10.7.1.2/32 address 10.7.1.1/32 {
        ike_id ike_sa_1;
        encapsulation_mode tunnel;
        lifetime 60 seconds;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                use_esn off;
        }
}
```

Initiator 2

```
general {
        listen 10.7.1.3;
        exchange_max_time 10;
        retransmits 3;
        spi_min 1000;
        spi_max 9999;
}
ike_sa address 10.7.1.1 {
        ike_id ike_sa_2;
        passive off;
        local_address 10.7.1.3;
        local_identifier user_fqdn init2@windriver.com;
        exchange_version 2;
        reauth no;
        lifetime 90 seconds;
        authentication {
                method pre_shared_key;
                shared_key shared_secret_2;
        }
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                hash_algorithm sha1;
                dh_group modp1024;
                use_esn off;
        }
}
ipsec_sa address 10.7.1.3/32 address 10.7.1.1/32 {
        ike_id ike_sa_2;
        encapsulation_mode tunnel;
        lifetime 60 seconds;
        proposal {
                encryption_algorithm des;
```

```
                integrity_algorithm sha1;
                use_esn off;
        }
}
```

Initiator 3

```
general {
        listen 10.7.1.4;
        exchange_max_time 10;
        retransmits 3;
        spi_min 1000;
        spi_max 9999;
}
ike_sa address 10.7.1.1 {
        ike_id ike_sa_3;
        passive off;
        local_address 10.7.1.4;
        local_identifier user_fqdn init3@windriver.com;
        exchange_version 2;
        reauth no;
        lifetime 90 seconds;
        authentication {
                method pre_shared_key;
                shared_key shared_secret_3;
        }
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                hash_algorithm sha1;
                dh_group modp1024;
                use_esn off;
        }
}
ipsec_sa address 10.7.1.4/32 address 10.7.1.1/32 {
        ike_id ike_sa_3;
        encapsulation_mode tunnel;
        lifetime 60 seconds;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm md5;
                use_esn off;
        }
}
```

Responder

```
general {
        listen 10.7.1.1;
        exchange_max_time 10;
        retransmits 3;
        spi_min 1000;
        spi_max 9999;
}
```

```
ike_sa address 0.0.0.0/0 {
        ike_id ike_sa_1;
        passive on;
        local_address 10.7.1.1;
        remote_identifier user_fqdn init1@windriver.com;
        exchange_version 2;
        reauth no;
        lifetime 110 seconds;
        authentication {
                method pre_shared_key;
                shared_key shared_secret_1;
        }
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                hash_algorithm sha1;
                dh_group modp1024;
                use_esn off;
        }
}
ike_sa address 0.0.0.0/0 {
        ike_id ike_sa_2;
        passive on;
        local_address 10.7.1.1;
        remote_identifier user_fqdn init2@windriver.com;
        exchange_version 2;
        reauth no;
        lifetime 110 seconds;
        authentication {
                method pre_shared_key;
                shared_key shared_secret_2;
        }
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                hash_algorithm sha1;
                dh_group modp1024;
                use_esn off;
        }
}
ike_sa address 0.0.0.0/0 {
        ike_id ike_sa_3;
        passive on;
        local_address 10.7.1.1;
        remote_identifier user_fqdn init3@windriver.com;
        exchange_version 2;
        reauth no;
        lifetime 110 seconds;
        authentication {
                method pre_shared_key;
                shared_key shared_secret_3;
        }
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                hash_algorithm sha1;
                dh_group modp1024;
```

```
                use_esn off;
        }
}
ipsec_sa address 10.7.1.1/32 address 0.0.0.0/0 {
        ike_id ike_sa_1;
        encapsulation_mode tunnel;
        lifetime 80 seconds;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm sha1;
                use_esn off;
        }
}
ipsec_sa address 10.7.1.1/32 address 0.0.0.0/0 {
        ike_id ike_sa_2;
        encapsulation_mode tunnel;
        lifetime 80 seconds;
        proposal {
                encryption_algorithm des;
                integrity_algorithm sha1;
                use_esn off;
        }
}
ipsec_sa address 10.7.1.1/32 address 0.0.0.0/0 {
        ike_id ike_sa_3;
        encapsulation_mode tunnel;
        lifetime 80 seconds;
        proposal {
                encryption_algorithm 3des;
                integrity_algorithm md5;
                use_esn off;
        }
}
```

5. Local Area Network Examples

Learn how to configure Wind River IKE for IPsec communication between two hosts on a LAN.

This section describes how to configure Wind River IKE for IPsec communication between two hosts on a LAN, as shown in the diagram below. West will be configured as the initiator and East as the responder. Authentication will use a preshared key.

The files can be found at the following locations:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike**_*x.x.x.x* **/ike/test/examples/initiator.cfg**

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike**_*x.x.x.x* **/ike/test/examples/responder.cfg**

Configuration File of West

The ike_sa section configures West as the initiator by means of the passive off entry. The SA proposal specifies 3DES for encryption and SHA1 for authentication and integrity checks. West will negotiate an IKE SA with East, using a preshared key for authentication. The **ipsec_sa** section specifies a flow that will protect all IP traffic between the hosts, using the ESP protocol with 3DES for encryption and SHA1 for authentication and integrity checks.

```
general {
                listen 10.1.2.100;
}
ike_sa address 10.1.2.200 {
                ike_id default;
                passive off;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
                }
                proposal {
                                dh_group 2;
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
}
ipsec_sa address 10.1.2.100 address 10.1.2.200 {
                ike_id default;
                proposal {
                                dh_group 2;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
}
```

Configuration File of East

responder.cfg, shown below, configures East as the responder. Note the following differences between the initiator and responder files:

- The addresses are different.
- The addresses in the **ike_sa** and **ipsec_sa** sections are reversed.
- The responder's passive entry is **on**, whereas the responder's is **off**.

```
general {
                listen 10.1.2.200;
}
ike_sa address 10.1.2.100 {
                ike_id default;
                passive on;
                authentication {
                                method pre_shared_key;
                                shared_key this_should_be_secret;
        }
        proposal {
                                dh_group 2;
```

```
                                hash_algorithm sha1;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
        }
}
ipsec_sa address 10.1.2.200 address 10.1.2.100 {
                ike_id default;
                proposal {
                                dh_group 2;
                                encryption_algorithm 3des;
                                integrity_algorithm sha1;
                }
}
```

Starting Wind River IKE on the Target on page 79 Start and test the connectivity of an IKE configuration.

5.1. Starting Wind River IKE on the Target

Start and test the connectivity of an IKE configuration.

Before You Begin

Before completing this exercise, you must have set up the configuration described in Local Area Network Examples on page 77.

About This Task

Perform the following procedures on both West and East.

Procedure

1. Boot the target.

   Download the VxWorks image and boot the target as described in Booting the Target on page 63.

2. Log on to the target.

   Log on to the target as described in Logging On to the Target on page 65.

3. Set the Negotiation Mode

Option	Description
Initiator Mode	In initiator mode (the default), the IKE task starts negotiating at startup. To configure initiator mode, set the initiator's passive entry to **off**.
Traffic Mode	In traffic mode, the IKE task starts negotiating when traffic is actually sent. 1. Set the initiator's passive entry to **on**.

Option	Description

	2. Configure Wind River IPsec to request an SA by issuing PF_KEY v2 Acquire or Require messages, by means of the keyadm command as shown below. Use the -acquire modifier if traffic is to be unprotected until the IPsec SA has been negotiated; use -require if traffic is to be blocked.
	
	Note: These commands do not constitute a complete IPsec configuration. For more information on IPsec configuration, see the *Wind River IPsec Programmer's Guide*.
	
	`keyadm addflow -spi 0 -dst 0.0.0.0 -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcflow 10.1.2.100 -dstflow 10.1.2.200 -require`
	
	`keyadm addflow -spi 0 -dst 0.0.0.0 -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcflow 10.1.2.200 -dstflow 10.1.2.100 -require -ingress`

4. Configure the IKE Task

   When Wind River IKE starts, the configuration file configures the IKE task. Perform the following steps to replace the default file with a custom file.

   1. Download the appropriate configuration file to the target using FTP. Replace the default configuration file /ram/ike.cfg with the new file.
   2. Switch from the C interpreter shell to the IPCOM shell using the cmd command.
   3. Verify the location of the configuration file.

```
sysvar list ipike.filename
ipike.filename=/ram/ike.cfg
```

5. Restart the IKE Task.

```
ipd start ipike
```

6. Check the connection by pinging West from East. If you configured IKE in traffic mode, this step triggers negotiation

```
ping 10.1.2.200
Pinging 10.1.2.200 (10.1.2.200) with 64 bytes of data:
Reply from 10.1.2.200 bytes=64 time=0ms ttl=64
Reply from 10.1.2.200 bytes=64 time=0ms ttl=64
Reply from 10.1.2.200 bytes=64 time=0ms ttl=64
```

```
Reply from 10.1.2.200 bytes=64 time=0ms ttl=64

--- 10.1.2.200 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 4000 ms
rtt min/avg/max = 0/0/0 ms
[vxWorks *]#
```

7. List the flows that have been created.

   The example output below shows three flows. The flow labelled [1] is a bypass flow on UDP port 500, which allows the IKE tasks to communicate with each other.

```
ipsecctrl flows
Flows:
[1] :: bypass src=0.0.0.0/0:500 dst=0.0.0.0/0:500 UDP >
SA: none
[2] :: use src=10.1.2.200/32:any dst=10.1.2.100/32:any any <
SA: ESP
spi=0xb18e81bc dst=10.1.2.100
[3] :: use src=10.1.2.100/32:any dst=10.1.2.200/32:any any >
SA: ESP
spi=0x80595296 dst=10.1.2.200
Total of 3 flows.
[vxWorks *]#
```

8. Dump the contents of the SADB to list the SAs that have been negotiated.

   The example output below shows two IPsec SAs.

```
keyadm dump
SADB_MSG: typ=DUMP err=0 sat=ESP len=176 seq=3 pid=0x3800f
SA: spi=80595296 sta=MATURE auth=MD5 enc=DES replay=0 fl=0x0
SRC: addr=10.1.2.100
DST: addr=10.1.2.200
CURRENT: addtime=1093249737 usetime=0 bytes=0
SOFT: addtime=27700 usetime=0 bytes=0
HARD: addtime=28000 usetime=0 bytes=0
SADB_MSG: typ=DUMP err=0 sat=ESP len=176 seq=0 pid=0x3800f
SA: spi=b18e81bc sta=MATURE auth=MD5 enc=DES replay=0 fl=0x0
SRC: addr=10.1.2.200
DST: addr=10.1.2.100
CURRENT: addtime=1093249737 usetime=0 bytes=0
SOFT: addtime=27700 usetime=0 bytes=0
HARD: addtime=28000 usetime=0 bytes=0
[vxWorks *]#
```

Parent topic: Local Area Network Examples on page 77

6. VPNC Interoperability Tests

Learn how to implement the test scenarios described in the Virtual Private Network (VPN) Consortium's interoperability specification.

The test network is a typical gateway-to-gateway VPN.

The test scenarios are as follows:

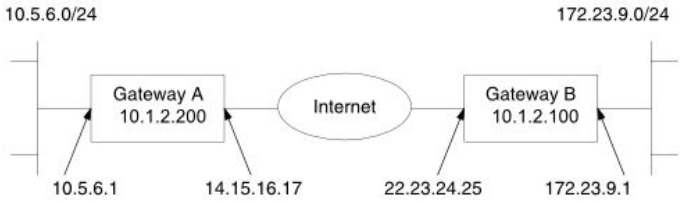
- Scenario 1: authentication using a preshared key
- Scenario 2: authentication using X.509 certificates

For each scenario, the following test cases are described:

- basic interoperability using 3DES encryption
- AES interoperability using AES encryption

The following diagram shows the network configuration used for all tests.

Figure 9. Network for Example Configuration Files



Gateway A connects the internal LAN 10.5.6.0/24 to the Internet. Its LAN interface has the address 10.5.6.1, and its WAN interface has the address 14.15.16.17.

Gateway B connects the internal LAN 172.23.9.0/24 to the Internet. Its WAN interface has the address 22.23.24.25. Its LAN interface, address 172.23.9.1, can be used for testing IPsec but is not needed for configuring Gateway A.

Configuration Files for Scenario 1 on page 82 Configuration files define the gateways to use a preshared key for authentication.
Configuration Files for Scenario 2 on page 84 Configuration files define the gateways to use X.509 keys for authentication.
Starting Wind River IKE on the Target on page 86 Start and test the connectivity of an IKE configuration for scenario 1 and scenario 2.

6.1. Configuration Files for Scenario 1

Configuration files define the gateways to use a preshared key for authentication.

Basic Interoperability Test

Phase 1 will be configured as follows:

- main mode negotiation
- 3DES encryption algorithm
- SHA-1 hashing algorithm
- MODP group 2 (1024 bits)
- preshared key "hr5xb84l6aa9r6"
- SA lifetime of 28800 seconds (eight hours) with zero Kb rekeying

Phase 2 will be configured as follows:

- 3DES encryption algorithm

- SHA-1 hashing algorithm
- ESP tunnel mode
- MODP group 2 (1024 bits)
- perfect forward secrecy for rekeying
- SA lifetime of 3600 seconds (one hour) with zero Kb rekeying
- selectors for all IP protocols and all ports, between 10.5.6.0/24 and 172.23.9.0/24, using IPv4 subnets

Configuration File of Gateway A

```
general {
                listen 14.15.16.17;
}
ike_sa address 22.23.24.25 {
        ike_id default;
        exchange_version 1;
        passive on;
        lifetime 28800 seconds;
        authentication {
                method pre_shared_key;
                shared_key hr5xb84l6aa9r6;
        }
        proposal {
                dh_group 2;
                hash_algorithm sha1;
                encryption_algorithm 3des;
                integrity_algorithm sha1;
        }
}
ipsec_sa address 10.5.6.0/24 address 172.23.9.0/24 {
        ike_id default;
        lifetime 3600 seconds;
        encapsulation_mode tunnel;
        proposal {
                dh_group 2;
                encryption_algorithm 3des;
                integrity_algorithm sha1;
        }
}
```

Configuration File of Gateway B

```
general {
                listen 22.23.24.25;
}
ike_sa address 14.15.16.17 {
        ike_id default;
        exchange_version 1;
        passive on;
        lifetime 28800 seconds;
        authentication {
                method pre_shared_key;
                shared_key hr5xb84l6aa9r6;
        }
```

```
        proposal {
                dh_group 2;
                hash_algorithm sha1;
                encryption_algorithm 3des;
                integrity_algorithm sha1;
        }
}
ipsec_sa address 172.23.9.0/24 address 10.5.6.0/24 {
        ike_id default;
        lifetime 3600 seconds;
        encapsulation_mode tunnel;
        proposal {
                dh_group 2;
                encryption_algorithm 3des;
                integrity_algorithm sha1;
        }
}
```

AES Interoperability Test

On both Gateway A and Gateway B, the Phase 1 and Phase 2 parameters are the same as for the Basic Interoperability Test, except for the encryption algorithm, which is 128-bit AES. In the configuration file, set encryption_algorithm to aes128 in the **ike_sa** and **ipsec_sa** sections.

Parent topic: VPNC Interoperability Tests on page 81

6.2. Configuration Files for Scenario 2

Configuration files define the gateways to use X.509 keys for authentication.

The Phase 1 parameters are the same as for the Basic Interoperability Test on page 82 using a preshared key, except for the authentication method, which is authentication with signatures authenticated by X.509 certificates. Both Gateway A and Gateway B have client certificates that chain to a root CA called Trusted Root CA.

The Phase 2 parameters are the same as for the Basic Interoperability Test on page 82 using a preshared key.

Certificates and Private Keys

For instructions on generating certificates, see http://www.openssl.org/docs. Attributes of the certificates used in this example are listed below.

Gateway A subject	`/C=SE/O=WindRiver/CN=Gateway A`
Gateway B subject	`/C=SE/O=WindRiver/CN=Gateway B`
Gateway A certificate and private key files	`/rsa_gwa.cer` `/rsa_gwa.key  (password 1111)`

Gateway B certificate and private key files	`/rsa_gwb.cer` `/rsa_gwb.key  (password 1111)`
Root CA certificate file	`/trusted/root_ca.cer`
Intermediate CA certificate file	`/intermediate/intermediate_ca.cer`

Configuration File of Gateway A

Start with the configuration file from Scenario 1. In the **ike_sa** section, add local_identifier and remote_identifier entries and change the **authentication** section, as shown below.

```
...
        local_identifier asn1_dn /C=SE/O=WindRiver/CN=Gateway A;
        remote_identifier asn1_dn /C=SE/O=WindRiver/CN=Gateway B;
        authentication {
                method rsa_signature;
                certificate rsa_gwa.cer;
                private_key rsa_gwa.key;
                private_key_password 1111;
                trusted_ca_store trusted;
                intermediate_ca_store intermediate;
        }
...
```

Configuration File of Gateway B

Start with the configuration file from Scenario 1. Add local_identifier and remote_identifier entries and change the **authentication** section, as shown below.

```
...
        local_identifier asn1_dn /C=SE/O=WindRiver/CN=Gateway B;
        remote_identifier asn1_dn /C=SE/O=WindRiver/CN=Gateway A;
        authentication {
                method rsa_signature;
                certificate rsa_gwb.cer;
                private_key rsa_gwb.key;
                private_key_password 1111;
                trusted_ca_store trusted;
                intermediate_ca_store intermediate;
        }
...
```

AES Interoperability Test

On both Gateway A and Gateway B, the Phase 1 and Phase 2 parameters are the same as for the Basic Interoperability Test, except for the encryption algorithm, which is 128-bit AES. In the configuration file, set encryption_algorithm to aes128 in the **ike_sa** and **ipsec_sa** sections.

Parent topic: VPNC Interoperability Tests on page 81

6.3. Starting Wind River IKE on the Target

Start and test the connectivity of an IKE configuration for scenario 1 and scenario 2.

About This Task

Perform the following procedures on both Gateway 1 and Gateway 2. The instructions apply to both Scenario 1 and Scenario 2.

Procedure

1. Boot the target.

   Download the VxWorks image and boot the target as described inBooting the Target on page 63.

2. Log on to the target.

   Log on to the target as described in Logging On to the Target on page 65.

3. Create routes.

   Create a route to handle traffic destined for addresses behind the other gateway.

   1. For gateway A:

   ```
   route add -net 172.23.9.0/24 22.23.24.25
   ```

   2. For gateway B:

   ```
   route add -net 10.5.6.0/24 14.15.16.17
   ```

4. Set the negotiation mode.

   You can configure the IKE task for initiator-mode or traffic-mode negotiation.

Option	Description
Initiator Mode	In initiator mode (the default), the IKE task automatically starts negotiating at startup. To configure initiator mode, edit the initiator's configuration file and set the passive entry to **off**.
Traffic Mode	In traffic mode, the IKE task starts negotiating when traffic is actually sent. To configure traffic mode, you must configure Wind River IPsec to request an SA by issuing a PF_KEY v2 Acquire or Require message. This can be done by setting the configuration parameter flowtype to acquire and to flowtype require in the ipsec_sa sections of the responder and initiator IKE configuration files. You can also do this by means of the keyadm command as shown below. Use the -acquire modifier if traffic is to

Option	Description

	be unprotected until the IPsec SA has been negotiated; use -require if traffic is to be blocked.
	
	Note: These commands do not constitute a complete IPsec configuration. For more information on IPsec configuration, see the *Wind River IPsec Programmer's Guide*.
	
	`keyadm addflow -spi 0 -dst 0.0.0.0 -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcflow 10.1.2.100 -dstflow 10.1.2.200 -require`
	
	`keyadm addflow -spi 0 -dst 0.0.0.0 -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcflow 10.1.2.200 -dstflow 10.1.2.100 -require -ingress`
	
	Gateway A:
	
	`> keyadm addflow -spi 0 -dst 0.0.0.0 -srcmask 255.255.255.0 -dstmask 255.255.255.0 -srcflow 10.5.6.0 -dstflow 172.23.9.0 -require`
	
	Gateway B:
	
	`> keyadm addflow -spi 0 -dst 0.0.0.0 -srcmask 255.255.255.0 -dstmask 255.255.255.0 -srcflow 172.23.9.0 -dstflow 10.5.6.0 -require`

5. Configure the IKE task.

   When Wind River IKE starts, the configuration file configures the IKE task. Perform the following steps to replace the default file with a custom file.

   1. Download the custom configuration file to the target using FTP. Replace the default configuration file **/ram/ike.cfg** with the new file.
   2. Switch from the C interpreter shell to the IPCOM shell using the cmd command.
   3. Verify the location of the configuration file.

   ```
   > sysvar list ipike.filename
   ipike.filename=/ram/ike.cfg
   ```

6. Restart the IKE task.

```
> ipd start ipike
```

7. Run tests and diagnostics.

   After you have started Wind River IKE, you can use the following shell commands to monitor either gateway:

   ike stat

   Print statistics.

   ike list

   List all IKE SAs.

   ipsecctrl sa

   List all IPsec SAs.

   ipsecctrl flows

   List all IPsec flows.

Parent topic: VPNC Interoperability Tests on page 81

7. ROHCoIPsec-Processed Packet Example

Learn how to configure ROHCoIPsec and verify that ROHC is enabled on the tunnel.

For more information on ROHCoIPsec, see Robust Header Compression over IPsec on page 16.

Configuration of ROHCoIPsec

For instructions on enabling ROHC in VxWorks, see the *Wind River Network Stack Programmer's Guide*. Once ROHC is enabled, it can be used in combination with IPsec.

To configure ROHCoIPsec, add an **rohc** section inside the **ipsec_sa** section.

For example:

```
rohc {
    profiles 4;
    integrity_algorithm md5,sha1,aes,sha256,sha384,sha512;
    icv_len 4;
}
```

In the above example, icv_len is optional. It must be greater than zero to enable the integrity value (ICV). Also note that, due to a limitation in the Wind River ROHC implementation, it is not possible to configure an asymmetric set of ROHC profiles. That is, you cannot configure the initiator with profile 2 and the responder with profile 4. The profile sets must be the same on both sides.

For more information on the rohc parameters, see ROHC Configuration Parameters on page 44.

Example

This example shows how to set up configuration for an initiator and a responder, examine the IPsec SAs, and verify that ROHC is enabled on the tunnel.

Initiator configuration:

```
general {
    listen 172.16.0.2;
}

ike_sa address 172.16.0.3 {
    ike_id default;
    local_address 172.16.0.2;
    exchange_version 2;
    exchange_mode aggressive;
    passive off;
    lifetime 28800 seconds;

    authentication {
        method pre_shared_key;
        shared_key this_should_be_secret;
    }

    proposal {
        dh_group modp1024;
        hash_algorithm sha1;
        encryption_algorithm 3des;
        integrity_algorithm sha1;
    }
}

ipsec_sa address 172.16.0.2 address 172.16.0.3 {
    ike_id default;
    lifetime 3600 seconds;
    encapsulation_mode tunnel;
    use_tfc off;
    proposal {
        dh_group modp1024;
        encryption_algorithm 3des;
        integrity_algorithm sha1;
        use_esn off;
    }

    rohc {
        profiles 4;
        integrity_algorithm sha1;
        icv_len 4;
    }
}
```

Responder configuration:

```
general {
    listen 172.16.0.3;
}
```

```
ike_sa address 172.16.0.2 {
    ike_id default;
    local_address 172.16.0.3;
    exchange_version 2;
    exchange_mode aggressive;
    passive on;
    lifetime 28800 seconds;

    authentication {
        method pre_shared_key;
        shared_key this_should_be_secret;
    }

    proposal {
        dh_group modp1024;
        hash_algorithm sha1;
        encryption_algorithm 3des;
        integrity_algorithm sha1;
    }
}

ipsec_sa address 172.16.0.3 address 172.16.0.2 {
    ike_id default;
    lifetime 3600 seconds;
    encapsulation_mode tunnel;
    use_tfc off;

    proposal {
        dh_group modp1024;
        encryption_algorithm 3des;
        integrity_algorithm sha1;
        use_esn off;
    }

    rohc {
        profiles 4;
        integrity_algorithm sha512;
        icv_len 2;
    }
}
```

When the tunnel is established, the IPsec SAs can be examined with the ipsecctrl command as follows:

```
#0> ipsecctrl sa
ipsecctrl sa
SA - Security Associations:
[ 1] :: ESP spi=0xbe0ef3ac src=172.16.0.3 dst=172.16.0.2 auth=sha1 hmac enc=esp3Des
-> ESP spi=0x78634388 src=172.16.0.2 dst=172.16.0.3
[ 2] :: ESP spi=0x78634388 src=172.16.0.2 dst=172.16.0.3 auth=sha1 hmac enc=esp3Des
Total of 2 SAs.
```

The rohc command can be used to verify that ROHC has been enabled on the tunnel. A psuedo interface is generated from the SPI of the inbound IPsec SA which always is unique.

In this case, it is 0xbe0ef3ac and the interface registered to be monitored inside ROHC is ipsecacf30ebe.

The compressor and decompressor contexts are created as soon as traffic that matches any of the added profiles flows through the tunnel.

```
#0> rohc
rohc
rohc> rohcshow
rohcshow
Supported profiles: uncompressed profile
UDP/IP profile
ESP/IP profile
IP profile
TCP/IP profile

Monitoring interfaces:
ipsecacf30ebe Supported profiles:
IP profile
Profile: IP profile
Compressor (2744d20)
Next to be used CID: 2
Compression Context (271ef40):
cid: 1
state: SO
mode: O-mode
decline_mode_trans: No
trans: Done
pkt_type: Compressed Type 0 Packet
ext_type: No extension
reject: No
num_ir: 0
num_compressed: 2
stream type: IPv4
src address: 172.16.0.2
dst address: 172.16.0.3
sn (network order): 53521
add_dynamic_ch ain: No
payload_offset: 162
ip_num: 1
Decompressor (273bf90)
Decompression Context (2748430):
cid: 3
state: Full Context
mode: O-mode
trans: Done
trans_mode: O-mode
first_ir: 0
num_decompressed: 3
num_failed: 0
crc: 15
stream type: IPv4
src address: 172.16.0.3
dst address: 172.16.0.2
sn (network order): 10
payload_offset: 146
ip_num: 1
Feedbacks: None

Maximum CID: 15
```

```
CID representation: Small CID
Feedback_for: Not defined
F_MAX_PERIOD: 256
F_MAX_TIME: 60

rohc> exit
exit
```

8. Setting Up IKE Between Two SBC8548 Boards

Run IKE between two SBC8548 boards.

About This Task

Use the following configuration to run IKE between two SBC8548 boards:

- Initiator: IP address 192.168.4.12 and IKE configuration file **initiator.cfg**.
- Responder: IP address 192.168.4.4 and IKE configuration file **responder.cfg**.

Below are the IKE configuration files used in the example.

```
#######################################################
# initiator.cfg - IKE Initiator configuration file example for host 1 (initiator)
# 192.168.4.4 = responder IP address
# 192.168.4.12 = initiator IP address
#######################################################
general {
        listen 192.168.4.12;
        exchange_max_time 300;
        spi_min 100000;
        spi_max 199999;
}
ike_sa address 192.168.4.4/24 {
        ike_id default;
        exchange_version 2; #1=IKEv1; 2=IKEv2
        exchange_mode = main; #IKEv1 only: main=Main Mode; aggressive=aggressive mode
        passive off; #on for Responder off for Initiator
        liveness 0;
        initial_contact off;
        lifetime 8 hours;
authentication {
        method pre_shared_key;
        shared_key this_should_be_secret;
}
proposal {
        dh_group 2,1;
        hash_algorithm sha1,md5;
        encryption_algorithm 3des,des;
        integrity_algorithm sha1,md5;
        }
}
ipsec_sa address 192.168.4.12/24 address 192.168.4.4/24 {
        ike_id default;
```

```
        encapsulation_mode transport;
        lifetime 3600 seconds;
        proposal {
                dh_group 2,1;
                encryption_algorithm 3des,des;
                integrity_algorithm sha1,md5;
        }
}
#######################################################
# END OF FILE
#######################################################


#######################################################
# responder.cfg - IKE Initiator configuration file example for host 2 (responder)
# 192.168.4.4 = responder IP address
# 192.168.4.12 = initiator IP address
#######################################################
general {
        192.168.4.4;
        exchange_max_time 300;
        spi_min 100000;
        spi_max 199999;
}
ike_sa address 192.168.4.12/24 {
        ike_id default;
        exchange_version 2; #1=IKEv1; 2=IKEv2
        exchange_mode = main; #IKEv1 only: main=Main Mode; aggressive=aggressive mode
        passive on; #on for Responder, off for Initiator
        liveness 0;
        initial_contact off;
        lifetime 8 hours;
authentication {
        method pre_shared_key;
        shared_key this_should_be_secret;
}
proposal {
        dh_group 2,1;
        hash_algorithm sha1,md5;
        encryption_algorithm 3des,des;
        integrity_algorithm sha1,md5;
        }
}
ipsec_sa address 192.168.4.4/24 address 192.168.4.12/24 {
        ike_id default;
        encapsulation_mode transport;
        lifetime 3600 seconds;
        proposal {
                dh_group 2,1;
                encryption_algorithm 3des,des;
                integrity_algorithm sha1,md5;
        }
}
#######################################################
# END OF FILE
#######################################################
```

Before You Begin

This procedure requires Wind River Platform for Consumer Devices, VxWorks Edition.

Procedure

1. Create a VxWorks Source Build (VSB) project based on the SBC8548 BSP.

   You can use Workbench or the wrtool command line tool. For more information about creating VSBs, see the *VxWorks 7 Configuration and Build Guide*.

2. Edit the Source Build Configuration to enable **IPNET_COREIP**, **IPNET_IPSECIKE**, and **OPENSSL**.

   Depending on your hardware, you might also want to enable hardware encryption/decryption by enabling **HWCRYPTO**.

3. Save and build the VSB.
4. Create a VxWorks Image Project based on the VSB you have created and built.
5. Include INCLUDE_INTERNET_KEY_EXCHANGE.

   Use the Kernel Configuration editor to locate **Data Security Components > > Data Security Protocols > Internet Key Exchange (IKE)**

   Note: You can also search for INCLUDE_INTERNET_KEY_EXCHANGE.

   This will resolve all required dependencies.

   For this example the following parameters and values are used in the initiator's main IKE configuration:

```
#define IPIKE_SOCK_BUF_SIZE "10000"
#define IPIKE_MAX_EXCHANGES "32"
#define IPIKE_PORT "500"
#define IPIKE_NAT_PORT "4500"
#define IPIKE_NAT_KEEPALIVE "20"
#define IPIKE_RESEND_DELAY "5"
#define IPIKE_CONFIG_FILE_NAME "initiator.cfg"
#define IPIKE_CONFIG_FILE_PATH "host:"
#define IPIKE_NONCE_SIZE "64"
#define IPIKE_CREATE_DEFAULT_CONFIG_FILE "0"
#define IPIKE_FLUSH_STARTUP "0"
#define IPIKE_FLUSH_SHUTDOWN "0"
#define IPIKE_FLUSH_RECONFIGURE "0"
#define IPIKE_DYNAMIC_ADDRESS_POOL_NETWORK4 "10.1.111.1/24"
#define IPIKE_DYNAMIC_ADDRESS_POOL_DNS4 "10.1.1.1"
#define IPIKE_DYNAMIC_ADDRESS_MODE "1"
```

   For this example, the responder's main IKE configuration includes the same parameters and values as in the initiator's configuration file, with the following exception:

```
#define IPIKE_CONFIG_FILE_NAME "responder.cfg"
```

   This configuration leaves the configuration files **initiator.cfg** and **responder.cfg** on the host. You can access them via netDrv through FTP. For this to work, the FTP client needs to be configured into VxWorks. Also, the device host: needs to be present. This is usually the case when VxWorks is downloaded through FTP by the boot ROM. The above configuration assumes that the

configuration files reside in the working directory of the FTP server. (One might change IPIKE_CONFIG_FILE_PATH to "/ram/" to put the configuration file in the local file system).

6. Adapt the IKE configuration files as needed (see below for a working example).

   Rebuild the vxWorks images and boot them. Execute on the responder:

   1. Restart the IKE daemon.

```
> cmd
[vxWorks *]# ipd kill ipike
[vxWorks *]# ipd start ipike
```

   This kills the IKE daemon task and then starts it again. You can also use:

```
[vxWorks *]# ipd reconfigure ipike
```

   2. Execute the same commands on the initiator:

```
> cmd
[vxWorks *]# ipd kill ipike
[vxWorks *]# ipd start ipike
```

   You now have created a secured communications channel.

   3. Call the following commands to check the current setting:

```
[vxWorks *]# sysvar list ipike
[vxWorks *]# keyadm dump
[vxWorks *]# ipsecctrl flows
```

   4. Call on the responder to check the secured communications.

```
[vxWorks *]# ping 192.168.4.12
```

   The ping should be successful. If you capture the ethernet packets between the two targets by using a tool like Ethereal, you should see that all packets sent between the two VxWorks targets are encrypted.

6. RUNNING SHELL COMMANDS

The ike shell command performs several operations.

These include:

- configuring the IKE task by loading a specified configuration file
- updating individual parameters of the configuration
- dumping the specified configuration file
- listing individual SAs or all SAs
- printing operational statistics for the IKE task

You can run the ike shell command at the IPCOM command prompt on the target device, or you can run it from a shell script. You can store the script on the target and execute it from a local process, or you can store it on an administrative computer that logs into the target (this approach would require a secure network).

For reference documentation on the ike shell command, see IKE Command Reference on page 96.

The ipd reconfigure ipike shell command restarts the IKE task and causes it to reread the configuration file, whereas commands such as ike config add *value* and ike config del *value* , or a call to the configuration API, affect only the specified parameters, leaving the rest unchanged.

Use the ipd reconfigure ipike shell command to reconfigure IKE whenever changes are made to the IKE configuration using the configuration API. Changes such as changing the listen addresses or interfaces also require a reconfigure.

Take care to flush (delete) invalid SAs if the addresses correspond to the changed addresses. IKE does not keep track of which SAs were negotiated over which interface and hence selective flushing using the ipd reconfigure ipike shell command is not possible. However, you can accomplish this using the PFKEYv2 interface to the SADB.

IKE Command Reference on page 96 Ike commands include config, list, stat, mobike and their subcommands.

1. IKE Command Reference

Ike commands include config, list, stat, mobike and their subcommands.

The ike shell command configures the IKE task by loading a specified configuration file. It can also update individual SAs, list all SAs, and print operational statistics for the task. You can change the name of this shell command by redefining *IPIKE_CMD_NAME* in the following file:

ike config add

Synopsis

```
ike config add value
```

Description

This command adds or replaces the specified data in the IKE configuration.

Parameters:

value

A string of configuration file content, consisting of a section name, entry name, and entry value. You need only specify the entries that are to be changed. For more information on configuration file entries, see IKE Configuration File on page 24.

```
ike_sa 10.1.1.1/32 {
    ike_id my_id;
}
```

Line breaks and white spaces are ignored, so you can specify the string on a single line as follows:

```
ike_sa 10.1.1.1/32 { ike_id my_id; }
```

Using IPCOM Shell for Long Strings

When a command string is too long to be entered in the IKE command shell, you can use the IPCOM shell command to enter long command line strings. To do so, you must set the *IPCOM_SHELL_CMD_STRING* variable in the file **ipcom_shell.c**. The *IPCOM_SHELL_CMD_STRING* variable sets the longest length string that the IPCOM shell can accept.

The **ipcom_shell.c** file is located in the following directory:

installDir **/vxworks-7/pkgs_v2/net/ipnet/coreip**_*x.x.x.x* **/src/ipcom/src/ipcom_shell.c**

The following is an example of a long command string that requires use of the IPCOM shell:

```
ike config add "ike_sa address 10.25.20.54 { ike_id defaulta; passive on;
authentication { method pre_shared_key; shared_key abc; } proposal { dh_group 2; encryption_al
gorithm 3des; hash_algorithm md5; integrity_algorithm md5; }
} ipsec_sa address 10.1.2.171 address 10.25.20.54 { ipsec_id default_1; ike_id defaulta; propo
sal { dh_group any,none;  encryption_algorithm any;
integrity_algorithm any;} } "
```

ike config delete

Synopsis

```
ike config delete value
```

Description

This command deletes the specified data from the IKE configuration. The command searches for the specified **ike_id** or **ipsec_id**, and if the identifier exists, the entire SA is deleted.

Parameters:

value

Configuration file content, as described for the ike config add command above.

Example

```
ike config delete "ike_sa address 192.168.199.43 { ike_id default; } "
```

ike config dump

Synopsis

```
ike config dump filename
```

Description

This command dumps the IKE configuration to the specified file.

Parameters:

filename

Name of the configuration file.

Example

```
ike config dump ike.cfg
```

ike config load

Synopsis

```
ike config load filename
```

Description

This command loads the IKE configuration from the specified configuration file.

filename

Name of the configuration file.

Example

```
ike config load ike.cfg
```

ike list

Synopsis

```
ike list
```

Description

This command lists all IKE SAs that currently exist. The output includes the state of each SA, which can be one of the following:

CONSTRUCTING

The SA is being negotiated.

ACTIVE

The SA can be used to protect traffic.

DYING

The SA has expired and will be deleted when all references are released.

Example output

```
[0] initiator cookie: 0x3a1f37747f6c6345
responder cookie: 0x03ca8c9296c78418
created 12 seconds ago as responder, ref.count: 2, state: ACTIVE
peer addr: 10.1.2.67, local addr: 10.1.2.130
sent bytes: 572, received bytes: 660
encryption algorithm: 3DES-CBC, hash algorithm: SHA1
authentication method: PRE_SHARED_KEY
```

```
life type: seconds, life duration: 14400
dh group: MODP_1024, generator: 2
```

ike stat

Synopsis

```
ike stat
```

Description

This command prints counts of successful and failed exchanges for the IKE task. The output includes the uptime, which is the number of seconds for which the task has been running. The exchange counts appear as the actual number of exchanges followed by the number per second in parentheses.

Example

```
Uptime 111 seconds
Successful exchanges:
Phase1: 0 (0.0)
Phase2: 0 (0.0)
Info: 0 (0.0)
Failed exchanges:
Phase1: 0 (0.0)
Phase2: 0 (0.0)
Info: 0 (0.0)
Number of active exchanges: 0
Number of active IKE SA: 0 (0.0)
```

ike mobike

Synopsis

`ike mobike` *old-address new-address*

Description

This command updates the IP address for an IKE SA.

Parent topic: Running Shell Commands on page 96

7. GLOSSARY

3DES

Triple-DES

AES

Advanced Encryption Standard

AH

The Authentication Header protocol provides connectionless integrity, data origin authentication, and an optional anti-replay service. Unlike ESP, AH secures parts of the IP header such as the source and destination addresses.

Authentication

In this guide, authentication refers to the process by which communicating hosts verify each other's identity.

CBC

Cipher Block Chaining

CCI

Common Cryptographic Interface

CTR

Counter encryption mode

Default policy

The *policy* applied to packets for which there is no matching policy in the Security Policy Database (SPD), usually a bypass or discard policy.

DES

Data Encryption Standard

EAP

Extensible Authentication Protocol

Encapsulation

The process of wrapping data in a protocol header. Specifically, the header and payload of one type of packet become the payload of another type of packet. For example, network traffic is wrapped in an Ethernet header before being transmitted over the network. When bridging dissimilar networks, the entire frame from one network is placed in the header used by the link layer protocol of the other network. See also *tunnel mode*.

ESN

Extended Sequence Number. ESN provides anti-replay support for high-speed IPsec implementations.

ESP

Encapsulating Security Payload may provide confidentiality (encryption), and limited traffic flow confidentiality. It also may provide connectionless integrity, data origin authentication, and an anti-replay service.

Flow

A flow is a 5-tuple set of packet selectors consisting of a source IP address, a destination IP address, a source port, a destination port, and an IP protocol identifier. In Wind River IKE, flows act as unique identifiers for sets of IPsec SA negotiation parameters (in the IKE configuration file) and for individual SAs (in the SADB).

HMAC

Keyed hashing for message authentication code

IKE

Internet Key Exchange is a protocol for exchanging keying material between two peers over an insecure network link. IKE is based on ISAKMP, Oakley and SKEME. There are two versions of IKE: version 1 (IKEv1), and version 2 (IKEv2).

IPCOM

Interpeak Common library containing utilities common to all Interpeak products.

IPsec

Internet Protocol security

ISAKMP

Internet Security Association and Key Management Protocol

lifetime

The configurable length of time after which IKE automatically deletes a protection suite, so that its repeated use does not pose a security risk.

MAC

Message authentication code

Oakley

Oakley is a free-form protocol that allows each party to advance the state of the protocol at its own speed. From Oakley, IKE borrowed the idea of different modes, each producing a similar result ñ an authenticated key exchange ñ through the exchange of information at different speeds.

Phase 1

Phase 1 of IKE negotiation is also referred to as “establishing an IKE SA”. In Phase 1, two ISAKMP peers negotiate an IKE SA, establishing a secure and authenticated channel with which to communicate. IKEv1 uses **Main Mode** and **Aggressive Mode** (currently not supported). In IKEv2 The **SA Init** method establishes the secure channel. **Main Mode** and **SA Init** send the authentication information in a set sequence, providing identity protection. **Aggressive Mode** does not provide identity protection because all the authentication information is sent at the same time. **Aggressive Mode** should only be used when network bandwidth is a concern.

Phase 2

Phase 2 of IKE negotiation is also referred to as “establishing an IPsec SA”. In Phase 2, the peers negotiate SAs on behalf of IPsec, establishing tunnels or endpoint SAs between IPsec hosts. In IKEv1, Phase 2 uses Quick Mode; in IKEv2 a Child SA is created because there is no need to repeat the authentication in by Phase 1.

PF_KEY v2

PF_KEY v2 is a generic key management API definition. Its purpose is to provide an OS-independent interface between key management applications, such as Wind River IKE, and OS-specific key engines, such as the security association database (SADB) of Wind River IPsec.

PFS

Perfect forward secrecy refers to the idea that compromise of a single key will permit access to only data protected by a single key. If the encryption key for phase 1 is compromised and PFS is not used, then it is possible to derive all keys that where exchanged in

phase 2 (providing that the attacker has all the packets from the exchange). Using PFS will slow down phase 2 exchanges since it requires one Diffie-Hellman operation per phase 2 exchange.

protection suite

A list of the security services that must be applied by a security protocol. All the services in the list must be applied. For example, the protection suite for the ESP protocol may consist of 3DES encryption, and the protection suite for the AH protocol may consist of keyed MD5 (HMAC).

policy

A rule for applying security processing to packets, for example, a rule that applies a specific type of encryption to packets of a specific traffic type that are destined for a specific port on a host at a specific address.

proposal

A proposal is a set of one or more transforms. All transforms in a proposal must be of the same type: AH or ESP.

RADIUS

Remote Authentication Dial In User Service

SA

A security association is a contract between two network peers, negotiated by IKE or set manually, that governs how the peers protect data passing between them. An SA applies to packets of a specific traffic type passing between specific ports on hosts at specific addresses. It determines the security protocol (AH or ESP), protocol mode, and protocol options to be used by IPsec for those packets. It is uniquely identified by a 3-tuple consisting of a Security Parameter Index (SPI), the destination address, and the protocol. An AH SA specifies the authentication key and the hash algorithm to be used. An ESP SA specifies the encryption key and the encryption algorithm to be used.

SA bundle

A security association bundle is a sequence of SAs by which traffic must be processed to satisfy a security policy.

SADB

Security Association Database

SHA-1

Secure Hashing Algorithm, version 1

SKEME

The secure key exchange mechanism defines a type of authenticated key exchange in which the parties use public key encryption to authenticate each other and share components of the exchange.

SPD

Security Policy Database

SPI

A security parameter index is a 32-bit value included in ESP and AH headers, which is used to identify which SA should be applied to a packet.

target

A CPU on which VxWorks runs.

TFC

Traffic Flow Confidentiality. A security mechanism for protecting identifying information in IPsec packets.

Transport mode

An end-to-end IPsec mode that protects upper-layer protocols. The end-point computers do the security processing.

Tunnel mode

A portal-to-portal IPsec mode that protects entire IP packets. A single node provides security to several computers. Tunnel mode employs the technique of encapsulation, which enables a packet from one network to be transported by another network, by adding an outer IP header that is accepted by the second network.